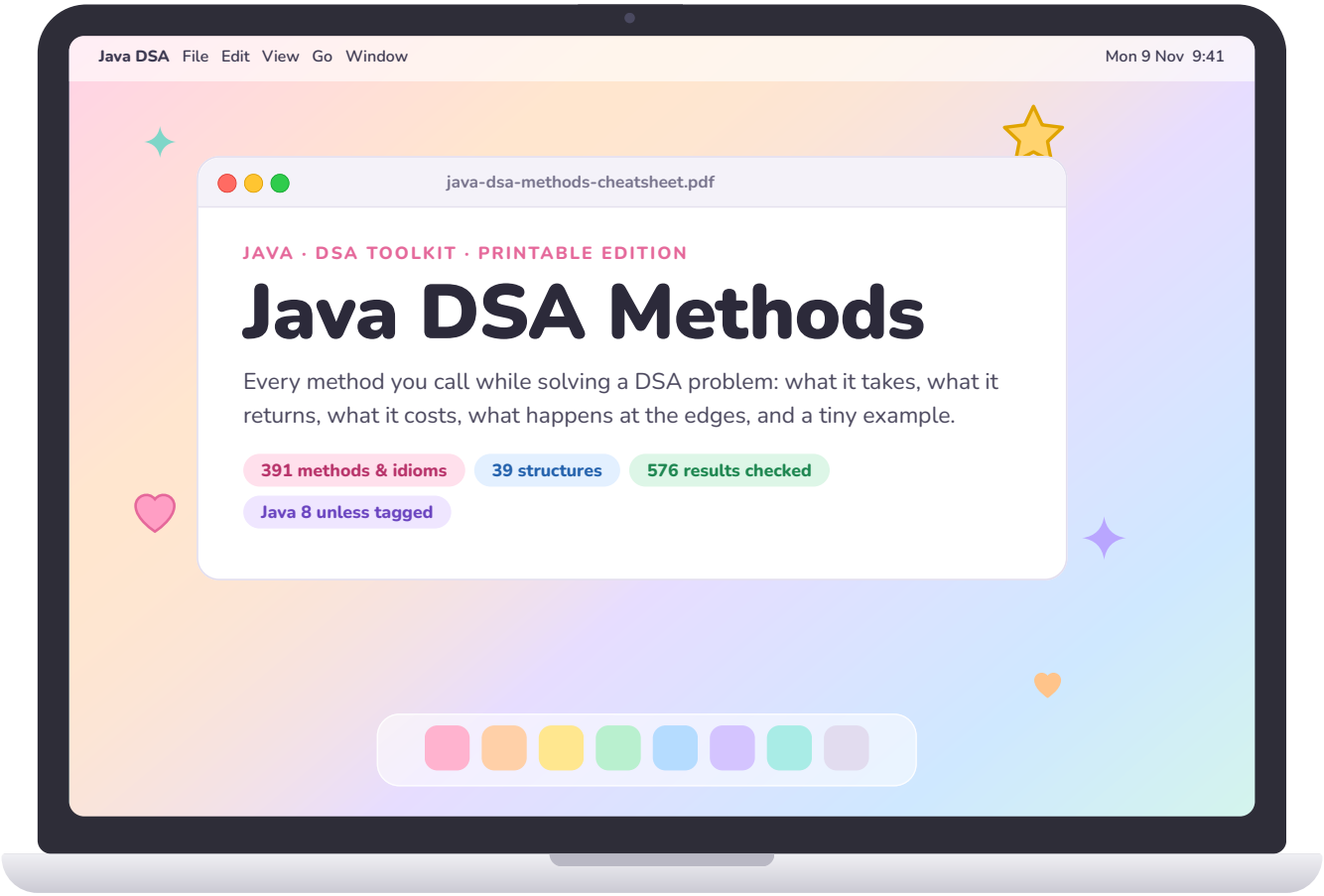


Numbers & bits page 4

Arrays & strings page 9

Lists page 14

Maps & sets page 19

Stacks, queues & heaps page 25

Ordering & streams page 30

Build it yourself page 33

Input & output page 38

How to read a row

`sig` is the call you type. *takes* = parameter types, *returns* = result type. `E` element, `K` / `V` key/value, `T` any object.

Examples start from the window's *start* state. `// → 5` value of the line · `// list → [5, 3]` state after it · `// throws X` it throws.

Cost colours

Time per call, *n* = current size.

`O(1)` `O(log n)` `O(n)` `O(n log n)`

"Amortised": an occasional slow call, averaged over many fast ones.

Java versions

Untagged = works in Java 8. A `JAVA 11+` tag needs that version or newer: check what the test platform runs.

Every example was compiled and run on OpenJDK 25.0.2; untagged ones also with `--release 8`.

Contents

40 items

FAVOURITES

- Start here
- Numbers & bits
- Arrays & strings
- Lists
- Maps & sets
- Stacks, queues & heaps
- Ordering & streams
- Build it yourself
- Input & output
- Before you submit

TAGS

`O(1)` `O(log n)` `O(n)` `O(n log n)` `JAVA 11+`

Name	Kind	Entries	Page
Primitive types and their limits	Numbers & bits	–	4
Integer arithmetic, casting and overflow	Numbers & bits	16	4
Integer, Long and Double	Numbers & bits	26	5
Character	Numbers & bits	10	6
char arithmetic	Numbers & bits	9	7
Math	Numbers & bits	15	7
Bit manipulation	Numbers & bits	16	8
Arrays: the language part	Arrays & strings	8	9
java.util.Arrays	Arrays & strings	15	10
String	Arrays & strings	27	11
StringBuilder	Arrays & strings	14	13
Collection<E>: methods every list, set and queue has	Lists	14	14
ArrayList<E> and List<E>	Lists	18	15
LinkedList<E>	Lists	5	17
java.util.Collections	Lists	13	17
Iterator<E> and ListIterator<E>	Lists	5	18
HashMap<K, V> and Map<K, V>	Maps & sets	26	19
LinkedHashMap and LinkedHashSet	Maps & sets	5	21
TreeMap<K, V>: a sorted map	Maps & sets	15	22
HashSet<E> and Set<E>	Maps & sets	11	23
TreeSet<E>: a sorted set	Maps & sets	12	24
Stack: ArrayDeque used as a stack	Stacks, queues & heaps	6	25
java.util.Stack (legacy)	Stacks, queues & heaps	6	26
Queue: ArrayDeque used as a queue	Stacks, queues & heaps	7	27
Deque: both ends	Stacks, queues & heaps	8	28
PriorityQueue<E>: a binary heap	Stacks, queues & heaps	9	29
Comparator<T> and Comparable<T>	Ordering & streams	7	30
Streams	Ordering & streams	11	31
BitSet	Ordering & streams	8	32
Converting between types	Ordering & streams	20	32
ListNode: singly linked list	Build it yourself	3	33
TreeNode: binary tree	Build it yourself	4	34
Graphs: adjacency lists, DFS, Dijkstra, topological order	Build it yourself	–	35
Union-Find (disjoint set union)	Build it yourself	4	36
Trie (prefix tree)	Build it yourself	3	37
Fenwick tree (binary indexed tree)	Build it yourself	3	37
Scanner	Input & output	4	38
BufferedReader and StringTokenizer	Input & output	4	38
PrintWriter and fast output	Input & output	4	39
Traps, one line each	Before you submit		40

START HERE

Which structure answers which question

Each structure answers a few questions in one fast step. Decide which question your algorithm asks most often, then pick the structure that answers it.

THE QUESTION	STRUCTURE	THE CALLS	COST
Have I seen *x*?	`HashSet<Integer>`	`add`, `contains`	`O(1)`
How many times did *x* appear? Where was *x*?	`HashMap<Integer, Integer>`, or `int[26]` for letters	`merge`, `getOrDefault`, `cnt[c - 'a']++`	`O(1)`
What was added most recently?	`ArrayDeque` as a stack	`push`, `pop`, `peek`	`O(1)`
What was added earliest?	`ArrayDeque` as a queue	`offer`, `poll`, `peek`	`O(1)`
What is the smallest (or largest) item so far?	`PriorityQueue`	`add`, `poll`, `peek`	`O(log n)`
What is the nearest key below or above *x*?	`TreeMap` / `TreeSet`	`floorKey`, `ceilingKey`, `lower`, `higher`	`O(log n)`
Smallest and largest, with deletions of any element?	`TreeMap` multiset	`firstKey`, `lastKey`, `merge`	`O(log n)`
What is element *i*?	array or `ArrayList`	`a[i]`, `get(i)`	`O(1)`
Maximum or minimum of a sliding window?	`ArrayDeque` monotonic deque	`offerLast`, `pollLast`, `peekFirst`, `pollFirst`	`O(1) amortised`
Next greater or smaller element?	monotonic stack	`push`, `pop`, `peek`	`O(1) amortised`
Are *x* and *y* in the same group, as groups merge?	Union-Find	`find`, `union`	`O(α(n))`
Is there a word with this prefix?	Trie	`insert`, `startsWith`	`O(length)`
Sum of `a[l..r]`?	prefix-sum array; Fenwick tree if values change	`p[r + 1] - p[l]`; `add`, `rangeSum`	`O(1)` / `O(log n)`
Keys in insertion order, or least recently used?	`LinkedHashMap`	`put`, `get`, `removeEldestEntry`	`O(1)`
Build a long string piece by piece?	`StringBuilder`	`append`, `setLength`, `toString`	`O(1) amortised`

START HERE

Costs at a glance

The calls that are easy to use by accident in a loop are the O(n) ones: `ArrayList.contains`, `ArrayList.remove(0)`, `PriorityQueue.remove(x)`, `PriorityQueue.contains(x)`, `String.substring` and `s += x`.

STRUCTURE	O(1)	O(LOG N)	O(N)
`int[]`	`a[i]`, `a[i] = x`, `a.length`	—	`Arrays.fill`, `copyOf`, `equals`; `sort` is O(n log n)
`ArrayList`	`get`, `set`, `size`, `add` at end (amortised), `remove` last	—	`add(i, x)`, `remove(i)`, `remove(Object)`, `contains`, `indexOf`
`HashMap`, `HashSet`	`get`, `put`, `merge`, `containsKey`, `add`, `contains`, `remove` (expected)	—	`containsValue`, iterating
`TreeMap`, `TreeSet`	`size`	`get`, `put`, `remove`, `floorKey`, `ceilingKey`, `firstKey`, `pollFirst`	iterating; `size()` of a sub-view
`ArrayDeque`	`push`, `pop`, `peek`, `offer`, `poll`, `offerFirst/Last`, `pollFirst/Last`	—	`contains`, `remove(Object)`
`PriorityQueue`	`peek`, `size`	`add`, `offer`, `poll`	`remove(Object)`, `contains`; building from a collection
`String`	`length`, `charAt`	—	`substring`, `indexOf`, `equals`, `split`, `+`, `toCharArray`
`StringBuilder`	`append` (amortised), `charAt`, `setCharAt`, `deleteCharAt` last, `length`	—	`insert`, `deleteCharAt(0)`, `reverse`, `toString`

PART 1

Numbers, characters and bits

Primitive types and their limits · Integer arithmetic, casting and overflow · Integer, Long and Double · Character · char arithmetic · Math · Bit manipulation

NUMBERS & BITS Primitive types and their limits

Pick the type from the largest value the problem can produce, not from the input size. With n ≤ 10^5 values up to 10^9, a sum can reach 10^{14}, which needs `long`.

TYPE	BITS	RANGE	DEFAULT IN AN ARRAY	LITERAL
`byte`	8	−128 … 127	`0`	`(byte) 5`
`short`	16	−32,768 … 32,767	`0`	`(short) 5`
`int`	32	−2,147,483,648 … 2,147,483,647 (about ±2.1 × 10^9)	`0`	`42`, `1_000_000`, `0b1010`, `0xff`
`long`	64	about ±9.2 × 10^{18}	`0L`	`42L`
`float`	32	about 7 significant digits	`0.0f`	`1.5f`
`double`	64	about 15–16 significant digits	`0.0`	`1.5`, `1e9`
`char`	16	0 … 65,535 (unsigned; a UTF-16 code unit)	`'\u0000'`	`'a'`, `'\n'`
`boolean`	—	`true` / `false`	`false`	`true`

Generics such as `List<E>` and `Map<K, V>` only take object types, so you write the **wrapper**: `Integer`, `Long`, `Character`, `Boolean`, `Double`. Java converts between `int` and `Integer` automatically (autoboxing), which costs time and memory and makes `null` possible.

TIP **"Infinity" that survives addition**

`Integer.MAX_VALUE + 1` wraps to a negative number, so a distance of `MAX_VALUE` breaks `dist[u] + w`. Use a smaller sentinel: `Integer.MAX_VALUE / 2`, `(int) 1e9`, or `Long.MAX_VALUE / 4` for `long`.

```
int INF = Integer.MAX_VALUE / 2;
int stillPositive = INF + INF;  // → 2147483646
long LINF = Long.MAX_VALUE / 4;
```

NUMBERS & BITS Integer arithmetic, casting and overflow

16 entries

Java never warns about overflow: `int` and `long` silently wrap around. Each row is an expression you can paste into a method; the result column is what Java prints.

START STATE · EVERY EXAMPLE BEGINS HERE

```
int a = 100_000, b = 100_000;
```

EXPRESSION	RESULT	WHY IT MATTERS
`7 / 2`	`3`	Integer division drops the fraction.
`-7 / 2`	`-3`	It truncates toward zero, not toward −∞. Use `Math.floorDiv` for floor.
`-7 % 2`	`-1`	`%` keeps the sign of the left operand. Use `Math.floorMod` for a non-negative remainder.
`(double) 7 / 2`	`3.5`	Cast one operand *before* dividing to get a fractional result.
`(int) 3.99`	`3`	Casting a `double` to `int` truncates toward zero.
`(int) -3.99`	`-3`	Truncation toward zero again, not floor.
`a * b`	`1410065408`	`int × int` is computed in `int` and wraps: 10^{10} does not fit.
`(long) a * b`	`10000000000`	Cast an operand to `long` first, then multiply.
`(long) (a * b)`	`1410065408`	Too late: the product already overflowed before the cast.
`Integer.MAX_VALUE + 1`	`-2147483648`	Wraps to `MIN_VALUE`.
`(a + b) / 2`	`100000`	Midpoint; can overflow when `a + b` > $2^{31} - 1$.
`a + (b - a) / 2`	`100000`	Overflow-free midpoint for binary search.
`(a + b) >>> 1`	`100000`	Also overflow-free for non-negative values: the unsigned shift reads the wrapped sum correctly.

EXPRESSION	RESULT	WHY IT MATTERS
`1_000_000_007L * 1_000_000_007L % 1_000_000_007L`	`0`	Under a modulus near 10^9, multiply in `long` and take `%` after every product.
`0.1 + 0.2 == 0.3`	`false`	Doubles are approximate. Compare with a tolerance: `Math.abs(x - y) < 1e-9`.
`5 / 0.0`	`Infinity`	Dividing a `double` by zero gives `Infinity`; dividing an `int` by zero throws `ArithmeticException`.

NUMBERS & BITS

Integer, Long and Double

♥ 26 entries

Static helpers on the wrapper classes: parsing, printing in other bases, bit counts and safe comparison. `Long` has the same methods for `long` values.

IMPORT `java.lang (no import needed)`

PARSING AND PRINTING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Integer.parseInt(String s)` RETURNS `int` `O(length)`	Converts decimal text to an `int`. Accepts a leading `-` or `+`. Throws `NumberFormatException` for anything else, including `""`, spaces and values outside the `int` range.	`int n = Integer.parseInt("-42");  // → -42` `int bad = Integer.parseInt("4 2");  // throws NumberFormatException`
`Integer.parseInt(String s, int radix)` RETURNS `int` `O(length)`	Parses `s` in base `radix` (2 to 36).	`int v = Integer.parseInt("1011", 2);  // → 11` `int h = Integer.parseInt("ff", 16);  // → 255`

`Integer.valueOf(int i)` RETURNS `Integer` `O(1)`	Boxes an `int` (there is also `valueOf(String s)`). Values −128..127 come from a cache, so `==` happens to work for them and fails above. Compare boxed values with `equals`.	`Integer a = Integer.valueOf(1000), b = Integer.valueOf(1000);`
		`boolean same = a == b;  // → false`
		`boolean eq = a.equals(b);  // → true`

`Integer.toString(int i)` RETURNS `String` `O(digits)`	Decimal text of `i`; same as `String.valueOf(i)` and `"" + i`.	`String t = Integer.toString(-305);  // → "-305"`
`Integer.toString(int i, int radix)` RETURNS `String` `O(digits)`	Text of `i` in base `radix`, lower-case letters.	`String hex = Integer.toString(255, 16);  // → "ff"` `String bin = Integer.toString(5, 2);  // → "101"`
`Integer.toBinaryString(int i)` RETURNS `String` `O(32)`	Binary digits without leading zeros. A negative number prints all 32 bits (two's complement). Also `toHexString`, `toOctalString`.	`String b = Integer.toBinaryString(10);  // → "1010"` `String neg = Integer.toBinaryString(-1);  // → "11111111111111111111111111111111"`

BIT COUNTING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Integer.bitCount(int i)` RETURNS `int` `O(1)`	Number of 1 bits (population count).	`int ones = Integer.bitCount(13);  // → 3`
`Integer.highestOneBit(int i)` RETURNS `int` `O(1)`	Value of the highest set bit (a power of two); 0 for 0.	`int h = Integer.highestOneBit(40);  // → 32`
`Integer.lowestOneBit(int i)` RETURNS `int` `O(1)`	Value of the lowest set bit; the same as `i & -i`.	`int l = Integer.lowestOneBit(12);  // → 4`
`Integer.numberOfLeadingZeros(int i)` RETURNS `int` `O(1)`	Zero bits above the highest 1 bit; 32 for 0. `31 - numberOfLeadingZeros(i)` is $\lfloor \log_2 i \rfloor$ for $i > 0$.	`int z = Integer.numberOfLeadingZeros(1);  // → 31` `int log2 = 31 - Integer.numberOfLeadingZeros(40);  // → 5`
`Integer.numberOfTrailingZeros(int i)` RETURNS `int` `O(1)`	Zero bits below the lowest 1 bit, which is the index of the lowest set bit; 32 for 0.	`int t = Integer.numberOfTrailingZeros(8);  // → 3`
`Integer.reverse(int i)` RETURNS `int` `O(1)`	Reverses the order of all 32 bits (LeetCode 190, Reverse Bits).	`int r = Integer.reverse(1);  // → -2147483648`

COMPARING AND COMBINING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Integer.compare(int x, int y)` RETURNS `int` `O(1)`	Negative if `x < y`, 0 if equal, positive if `x > y`. Use it in comparators instead of `x - y`, which overflows for values far apart.	`int c = Integer.compare(3, 7);  // → -1` `int wrong = Integer.MIN_VALUE - 1;  // → 2147483647`

`Integer.sum(int a, int b)` RETURNS `int` `O(1)`	`a + b` as a method, mainly for method references: `map.merge(k, 1, Integer::sum)`. `Integer.max` and `Integer.min` work the same way.	`Map<Character, Integer> cnt = new HashMap<>();`
		`cnt.merge('a', 1, Integer::sum);`
		`cnt.merge('a', 1, Integer::sum);`
		`int n = cnt.get('a');  // → 2`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`intValue()` RETURNS `int` O(1)	Unboxes an `Integer` to `int` (also `longValue()`, `doubleValue()`). Unboxing `null`, explicitly or automatically, throws `NullPointerException`.	`Integer boxed = 7;`
		`int x = boxed.intValue(); // → 7`
		`Integer none = null;`
		`int y = none; // throws NullPointerException`

`equals(Object o)` RETURNS `boolean` O(1)	True only if `o` is an `Integer` with the same value. `Integer.valueOf(5).equals(5L)` is false because `5L` boxes to a `Long`.	`Integer a = 5;`
		`boolean t = a.equals(5); // → true`
		`boolean f = a.equals(5L); // → false`

`compareTo(Integer other)` RETURNS `int` O(1)	Natural-order comparison of two `Integer` objects; what `Collections.sort`, `TreeMap` and `PriorityQueue` use by default.	`Integer a = 5;` `int c = a.compareTo(9); // → -1`

CONSTANTS

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Integer.MAX_VALUE` TAKES field: no parentheses RETURNS `int` O(1)	2,147,483,647 = $2^{31} - 1$. Adding 1 wraps to `MIN_VALUE`.	`int big = Integer.MAX_VALUE; // → 2147483647` `int wrap = Integer.MAX_VALUE + 1; // → -2147483648`
`Integer.MIN_VALUE` TAKES field: no parentheses RETURNS `int` O(1)	−2,147,483,648 = -2^{31}. Its negation and `Math.abs` of it are still `MIN_VALUE`.	`int neg = -Integer.MIN_VALUE; // → -2147483648`

LONG AND DOUBLE

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Long.parseLong(String s)` RETURNS `long` O(length)	Parses a `long`. Use it when the input can exceed 2.1×10^9.	`long v = Long.parseLong("9000000000"); // → 9000000000`
`Long.MAX_VALUE` TAKES field: no parentheses RETURNS `long` O(1)	9,223,372,036,854,775,807 ≈ 9.2×10^{18}. `Long.MIN_VALUE` is -2^{63}.	`long m = Long.MAX_VALUE; // → 9223372036854775807`
`Long.bitCount(long i)` RETURNS `int` O(1)	The bit methods exist for `long` too: `bitCount`, `numberOfTrailingZeros`, `numberOfLeadingZeros`, `highestOneBit`, `toBinaryString`. Counts come back as `int`.	`int ones = Long.bitCount((1L << 40) - 1); // → 40`
`Long.compare(long x, long y)` RETURNS `int` O(1)	Safe comparison for `long` values, e.g. in a comparator over `long[]` pairs.	`int c = Long.compare(5L, 2L); // → 1`
`Double.parseDouble(String s)` RETURNS `double` O(length)	Parses a decimal number.	`double d = Double.parseDouble("3.5"); // → 3.5`
`Double.compare(double x, double y)` RETURNS `int` O(1)	Comparison for doubles; use it in comparators.	`int c = Double.compare(0.1 + 0.2, 0.3); // → 1`
`Double.MAX_VALUE` TAKES field: no parentheses RETURNS `double` O(1)	Largest finite double, about 1.8×10^{308}. `Double.MIN_VALUE` is the smallest **positive** double, not the most negative one: use `-Double.MAX_VALUE` or `Double.NEGATIVE_INFINITY` as a starting maximum.	`double tiny = Double.MIN_VALUE; // → 4.9E-324` `double lowest = -Double.MAX_VALUE; // → -1.7976931348623157E308`

NUMBERS & BITS Character

10 entries

Static tests and conversions for a single `char`. They also accept an `int` code point.

IMPORT `java.lang (no import needed)`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Character.isDigit(char ch)` RETURNS `boolean` O(1)	True for `'0'`–`'9'` (and digits of other scripts).	`boolean d = Character.isDigit('7'); // → true`
`Character.isLetter(char ch)` RETURNS `boolean` O(1)	True for letters of any alphabet, either case.	`boolean l = Character.isLetter('Q'); // → true` `boolean u = Character.isLetter('_'); // → false`
`Character.isLetterOrDigit(char ch)` RETURNS `boolean` O(1)	Letter or digit: the filter for palindrome problems that ignore punctuation (LeetCode 125).	`boolean p = Character.isLetterOrDigit(','); // → false`
`Character.isUpperCase(char ch)` RETURNS `boolean` O(1)	Also `isLowerCase`.	`boolean up = Character.isUpperCase('a'); // → false`
`Character.isWhitespace(char ch)` RETURNS `boolean` O(1)	Space, tab, newline and other whitespace.	`boolean w = Character.isWhitespace('\t'); // → true`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Character.toUpperCase(char ch)` RETURNS `char` O(1)	Upper-case form, or `ch` itself if it has none. Also `toLowerCase`.	`char c = Character.toUpperCase('b'); // → 'B'` `char same = Character.toUpperCase('7'); // → '7'`
`Character.getNumericValue(char ch)` RETURNS `int` O(1)	Digit value of `ch`, but letters count too: `'a'` gives 10, up to `'z'` = 35. For plain digits, `ch - '0'` is shorter and faster.	`int v = Character.getNumericValue('7'); // → 7` `int a = Character.getNumericValue('a'); // → 10`
`Character.digit(char ch, int radix)` RETURNS `int` O(1)	Value of `ch` as a digit in base `radix`, or −1 if it is not one.	`int f = Character.digit('f', 16); // → 15` `int none = Character.digit('9', 8); // → -1`
`Character.forDigit(int digit, int radix)` RETURNS `char` O(1)	Character for a digit value (lower case).	`char c = Character.forDigit(11, 16); // → 'b'`
`Character.toString(char c)` RETURNS `String` O(1)	One-character string; same as `String.valueOf(c)`.	`String t = Character.toString('x'); // → "x"`

NUMBERS & BITS char arithmetic

♥ 9 entries

A `char` is a 16-bit number, so you can subtract and add to it. Any arithmetic on a `char` produces an `int`; cast back with `(char)`.

START STATE · EVERY EXAMPLE BEGINS HERE

```
char c = 'e';
```

EXPRESSION	RESULT	WHY IT MATTERS
`c - 'a'`	`4`	Index 0–25 of a lower-case letter: the key for an `int[26]` counter.
`(char) ('a' + 3)`	`'d'`	Back from an index to a letter.
`'7' - '0'`	`7`	Digit character to its value.
`(char) ('0' + 5)`	`'5'`	Digit value to its character.
`(char) (c - 'a' + 'A')`	`'E'`	ASCII lower to upper case (only for `'a'`–`'z'`).
`(int) c`	`101`	The character code (ASCII `'e'` is 101). `int[128]` covers all ASCII.
`c + 1`	`102`	`char + int` is an `int`, not a `char`.
`(char) (c + 1)`	`'f'`	Cast to get the next letter.
`c == 'e'`	`true`	Compare `char` values with `==`. `'e'` (char) and `"e"` (String) are different types.

NUMBERS & BITS Math

♥ 15 entries

All methods are static. Most have overloads for `int`, `long`, `float` and `double`; the result has the widest argument type.

IMPORT `java.lang (no import needed)`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Math.max(int a, int b)` RETURNS `int` O(1)	Larger of two values; `Math.min` for the smaller. Mixing `int` and `long` returns `long`.	`int m = Math.max(3, 9); // → 9` `long big = Math.max(3, 10_000_000_000L); // → 10000000000`
`Math.abs(int a)` RETURNS `int` O(1)	Absolute value. `Math.abs(Integer.MIN_VALUE)` is still negative because $+2^{31}$ does not fit in an `int`.	`int a = Math.abs(-7); // → 7` `int trap = Math.abs(Integer.MIN_VALUE); // → -2147483648`
`Math.pow(double a, double b)` RETURNS `double` O(1)	a^b as a `double`. Cast to `long` for whole numbers, and only trust it up to about 2^{53}. For powers under a modulus, write fast exponentiation.	`long p = (long) Math.pow(2, 10); // → 1024` `double r = Math.pow(2, 0.5); // → 1.4142135623730951`
`Math.sqrt(double a)` RETURNS `double` O(1)	Square root as a `double`; `NaN` for negatives. For an integer square root, cast and then check `r * r`.	`int r = (int) Math.sqrt(17); // → 4` `boolean square = r * r == 17; // → false`
`Math.cbrt(double a)` RETURNS `double` O(1)	Cube root; works for negative numbers.	`double c = Math.cbrt(-27); // → -3.0`
`Math.floor(double a)` RETURNS `double` O(1)	Largest whole number ≤ `a`, still as a `double`. `Math.ceil` gives the smallest whole number ≥ `a`.	`double f = Math.floor(-2.5); // → -3.0` `double c = Math.ceil(2.1); // → 3.0`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Math.round(double a)` RETURNS `long` O(1)	Nearest whole number as a `long` (an `int` for a `float` argument). Halves round up, toward +∞.	`long r = Math.round(2.5); // → 3` `long n = Math.round(-2.5); // → -2`
`Math.floorDiv(int x, int y)` RETURNS `int` O(1)	Division rounded toward −∞; `/` rounds toward 0. Also for `long`.	`int q = Math.floorDiv(-7, 2); // → -4` `int t = -7 / 2; // → -3`
`Math.floorMod(int x, int y)` RETURNS `int` O(1)	Remainder with the sign of `y`, so the result is in 0..y−1 for positive `y`. Use it for cyclic indexes and modular arithmetic with negative numbers.	`int m = Math.floorMod(-7, 5); // → 3` `int r = -7 % 5; // → -2`
`Math.addExact(int x, int y)` RETURNS `int` O(1)	Sum that throws `ArithmeticException` on overflow instead of wrapping. Also `subtractExact`, `multiplyExact`, `negateExact`, and `long` versions.	`int s = Math.addExact(2, 3); // → 5` `int o = Math.multiplyExact(100_000, 100_000); // throws ArithmeticException`
`Math.toIntExact(long value)` RETURNS `int` O(1)	Narrows a `long` to `int`, throwing `ArithmeticException` if it does not fit.	`int i = Math.toIntExact(123L); // → 123`
`Math.log(double a)` RETURNS `double` O(1)	Natural logarithm; `Math.log10` for base 10. For $\log_2$ of an integer, `31 - Integer.numberOfLeadingZeros(x)` is exact, while `Math.log(x) / Math.log(2)` can be off near powers of two.	`double l = Math.log10(1000); // → 3.0` `double e = Math.log(Math.E); // → 1.0`
`Math.hypot(double x, double y)` RETURNS `double` O(1)	$\sqrt{x^2 + y^2}$ without overflow in the squares; Euclidean distance.	`double h = Math.hypot(3, 4); // → 5.0`

`Math.random()` RETURNS `double` O(1)	Uniform `double` in [0, 1). For whole numbers use `new Random().nextInt(n)`, which returns 0..n−1.	`double r = Math.random();`
		`boolean inRange = r >= 0 && r < 1; // → true`
		`int dice = new Random(7).nextInt(6) + 1;`

`Math.ceilDiv(int x, int y)` JAVA 18+ RETURNS `int` O(1)	Division rounded toward +∞. In Java 8, for positive numbers, write `(x + y - 1) / y`.	`int c = Math.ceilDiv(7, 2); // → 4` `int java8 = (7 + 2 - 1) / 2; // → 4`

Number-theory helpers you write yourself

TEMPLATE

```
static long gcd(long a, long b) { return b == 0 ? a : gcd(b, a % b); }

static long power(long base, long exp, long mod) {      // base^exp % mod in O(log exp)
    long result = 1;
    base %= mod;
    while (exp > 0) {
        if ((exp & 1) == 1) result = result * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return result;
}

long g = gcd(12, 18);  // → 6
long l = 12 / gcd(12, 18) * 18;  // → 36
long p = power(2, 30, 1_000_000_007L);  // → 73741817
```

`Math` has no gcd or modular power. These two are short enough to type from memory. Divide before multiplying in the lcm to avoid overflow.

NUMBERS & BITS Bit manipulation

16 entries

Operators: `&` and, `|` or, `^` xor, `~` not, `<<` shift left, `>>` shift right keeping the sign, `>>>` shift right filling with zeros. Bit i means the bit worth 2^i, counting from 0 on the right.

START STATE · EVERY EXAMPLE BEGINS HERE

```
int x = 12;   // binary 1100
```

EXPRESSION	RESULT	WHY IT MATTERS
`x & 1`	0	1 if `x` is odd, 0 if even.
`(x >> 2) & 1`	1	Reads bit 2.
`x | (1 << 0)`	13	Sets bit 0.
`x & ~(1 << 2)`	8	Clears bit 2.
`x ^ (1 << 3)`	4	Toggles bit 3.
`x & -x`	4	Keeps only the lowest set bit (Fenwick trees use this).
`x & (x - 1)`	8	Drops the lowest set bit; loop on it to count bits.

EXPRESSION	RESULT	WHY IT MATTERS
`x > 0 && (x & (x - 1)) == 0`	`false`	Power-of-two test. `&` binds weaker than `==`, so the parentheses are required.
`x << 1`	`24`	Multiplies by 2.
`x >> 1`	`6`	Divides by 2, rounding toward −∞ for negative numbers.
`-12 >> 1`	`-6`	`>>` keeps the sign.
`-12 >>> 28`	`15`	`>>>` fills with zeros, so negative numbers become large positive ones.
`~x`	`-13`	Bitwise not; always equals `-x - 1`.
`x ^ x`	`0`	A value xor itself is 0, and `x ^ 0 == x`: xor-ing a whole array leaves the element that appears an odd number of times.
`1 << 31`	`-2147483648`	Shifting into bit 31 gives a negative `int`.
`1L << 40`	`1099511627776`	Use a `long` literal for shifts of 31 or more; `1 << 40` silently equals `1 << 8`.

Enumerating subsets with a bitmask

TEMPLATE

```
int[] items = {3, 5, 7};
int n = items.length;
List<Integer> sums = new ArrayList<>();
for (int mask = 0; mask < (1 << n); mask++) {          // every subset, 2^n of them
    int sum = 0;
    for (int i = 0; i < n; i++)
        if ((mask >> i & 1) == 1) sum += items[i];      // is item i in this subset?
    sums.add(sum);
}
// sums → [0, 3, 5, 8, 7, 10, 12, 15]

int m = 0b1011;
List<Integer> subs = new ArrayList<>();
for (int sub = m; sub > 0; sub = (sub - 1) & m) subs.add(sub);   // every non-empty submask of m
// subs → [11, 10, 9, 8, 3, 2, 1]
```

Bitmask DP and brute force over subsets use these two loops. `2^n` is fine up to about n = 20.

PART 2

Arrays and strings

Arrays · java.util.Arrays · String · StringBuilder

ARRAYS & STRINGS — Arrays: the language part

♥ 8 entries

An array has a fixed length chosen at creation. Reading and writing by index is O(1), which makes `int[]` the fastest structure in Java: prefer it to a `HashMap` whenever the keys are small integers.

START STATE · EVERY EXAMPLE BEGINS HERE

```
int[] a = {5, 3, 9, 1};
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`new int[n]` TAKES `int n (the length)` RETURNS `int[]` `O(n)`	New array of length `n` filled with defaults: 0 for numbers, `false` for `boolean`, `'\u0000'` for `char`, `null` for objects.	`int[] z = new int[3];  // → [0, 0, 0]` `String[] names = new String[2];  // → [null, null]`
`{v0, v1, ...}` TAKES `the element values` RETURNS `int[]` `O(n)`	Array literal. The short form only works in a declaration; elsewhere write `new int[] {...}`.	`int[] b = {1, 2, 3};` `b = new int[] {4, 5};  // b → [4, 5]`
`a.length` TAKES field: no parentheses RETURNS `int` `O(1)`	Number of elements. Arrays use the **field** `length`; `String` has the method `length()`; collections have `size()`.	`int n = a.length;  // → 4`

`a[i]`
TAKES `int index`
RETURNS `int` `O(1)`
	Reads or writes element `i` (0-based). Outside 0..length−1 it throws `ArrayIndexOutOfBoundsException`.	`int first = a[0];  // → 5`
		`a[1] = 7;  // a → [5, 7, 9, 1]`
		`int bad = a[4];  // throws ArrayIndexOutOfBoundsException`

`a.clone()` RETURNS `int[]` `O(n)`	Independent copy of a 1-D array. On a 2-D array it copies only the outer array; the rows stay shared.	`int[] c = a.clone();` `c[0] = 100;  // a → [5, 3, 9, 1]`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`new int[rows][cols]`
TAKES `int rows, int cols`
RETURNS `int[][]` `O(rows·cols)`
	A grid with every cell 0. `g.length` is the number of rows and `g[0].length` the number of columns.	`int[][] g = new int[2][3];`
		`int rows = g.length; // → 2`
		`int cols = g[0].length; // → 3`

`new int[rows][]`
TAKES `int rows`
RETURNS `int[][]` `O(rows)`
	Jagged array: each row is `null` until you assign it, and rows can have different lengths.	`int[][] tri = new int[3][];`
		`tri[0] = new int[1];`
		`tri[2] = new int[3];`
		`// tri → [[0], null, [0, 0, 0]]`

`for (int x : a)`
TAKES `an array or any Iterable`
RETURNS not a call `O(n)`
	For-each loop over every element in order. You do not get the index, and assigning to `x` does not change the array.	`int sum = 0;`
		`for (int x : a) sum += x;`
		`// sum → 18`

PITFALL **Printing or comparing an array directly**

`System.out.println(a)` prints a type tag and hash such as `[I@1b6d3586`, and `a.equals(b)` compares references. Use `Arrays.toString(a)` and `Arrays.equals(a, b)`.

```
int[] b = {5, 3, 9, 1};
boolean ref = a.equals(b);  // → false
boolean content = Arrays.equals(a, b);  // → true
```

ARRAYS & STRINGS

java.util.Arrays

15 entries

Static helpers that work on arrays: sorting, filling, copying, searching and printing.

IMPORT `java.util.Arrays`

START STATE · EVERY EXAMPLE BEGINS HERE

```
int[] a = {5, 3, 9, 1};
```

SORTING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Arrays.sort(int[] a)` RETURNS `void` `O(n log n)`	Sorts ascending, in place. Works for every primitive array type (`long[]`, `char[]`, `double[]`, …). There is no descending sort for primitive arrays.	`Arrays.sort(a); // a → [1, 3, 5, 9]`
`Arrays.sort(int[] a, int fromIndex, int toIndex)` RETURNS `void` `O(k log k)`	Sorts only indexes fromIndex..toIndex−1.	`Arrays.sort(a, 0, 3); // a → [3, 5, 9, 1]`

`Arrays.sort(T[] a, Comparator<? super T> c)`
RETURNS `void` `O(n log n)`
	Sorts an array of **objects** by a comparator; stable. This is how you sort descending (box to `Integer[]` first) or sort the rows of an `int[][]`.	`Integer[] b = {5, 3, 9, 1};`
		`Arrays.sort(b, Collections.reverseOrder()); // b → [9, 5, 3, 1]`
		`int[][] iv = {{3, 4}, {1, 2}, {2, 6}};`
		`Arrays.sort(iv, (x, y) -> Integer.compare(x[0], y[0])); // iv → [[1, 2], [2, 6], [3, 4]]`

FILLING AND COPYING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`Arrays.fill(int[] a, int val)`
RETURNS `void` `O(n)`
	Sets every element to `val`; `fill(a, from, to, val)` for a range. For a 2-D array, fill each row in a loop.	`Arrays.fill(a, -1); // a → [-1, -1, -1, -1]`
		`int[][] dp = new int[2][3];`
		`for (int[] row : dp) Arrays.fill(row, 7);`
		`// dp → [[7, 7, 7], [7, 7, 7]]`

`Arrays.copyOf(int[] original, int newLength)` RETURNS `int[]` `O(n)`	New array of `newLength`: copies the elements, then pads with defaults or truncates.	`int[] longer = Arrays.copyOf(a, 6); // → [5, 3, 9, 1, 0, 0]` `int[] shorter = Arrays.copyOf(a, 2); // → [5, 3]`
`Arrays.copyOfRange(int[] original, int from, int to)` RETURNS `int[]` `O(to - from)`	New array with indexes from..to−1.	`int[] mid = Arrays.copyOfRange(a, 1, 3); // → [3, 9]`
`System.arraycopy(Object src, int srcPos, Object dest, int destPos, int length)` RETURNS `void` `O(length)`	Copies `length` elements into an **existing** array. Overlapping ranges are handled correctly.	`int[] d = new int[5];` `System.arraycopy(a, 0, d, 1, 4); // d → [0, 5, 3, 9, 1]`
`Arrays.setAll(int[] array, IntUnaryOperator generator)` RETURNS `void` `O(n)`	Sets `array[i] = generator(i)` for every index.	`int[] sq = new int[4];` `Arrays.setAll(sq, i -> i * i); // sq → [0, 1, 4, 9]`

SEARCHING AND COMPARING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`Arrays.binarySearch(int[] a, int key)` RETURNS `int` O(log n)	Searches a **sorted** array. Returns an index of `key`, or `-(insertionPoint) - 1` if absent. With duplicates it returns *any* matching index, not necessarily the first; write your own lower bound when the first one matters.	`int[] s = {1, 3, 5, 9};`
		`int i = Arrays.binarySearch(s, 5);  // → 2`
		`int miss = Arrays.binarySearch(s, 4);  // → -3`
		`int insertAt = -miss - 1;  // → 2`

`Arrays.equals(int[] a, int[] b)` RETURNS `boolean` O(n)	True if both have the same length and equal elements. Use `Arrays.deepEquals` for 2-D arrays.	`boolean same = Arrays.equals(a, new int[] {5, 3, 9, 1});  // → true`
`Arrays.compare(int[] a, int[] b)` JAVA 9+ RETURNS `int` O(n)	Lexicographic comparison: negative, 0 or positive.	`int c = Arrays.compare(new int[] {1, 2}, new int[] {1, 3});  // → -1`

PRINTING AND CONVERTING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Arrays.toString(int[] a)` RETURNS `String` O(n)	Readable text such as `[5, 3, 9, 1]`.	`String t = Arrays.toString(a);  // → "[5, 3, 9, 1]"`
`Arrays.deepToString(Object[] a)` RETURNS `String` O(n)	Text of a 2-D (or deeper) array.	`int[][] g = {{1, 2}, {3, 4}};` `String t = Arrays.deepToString(g);  // → "[[1, 2], [3, 4]]"`

`Arrays.asList(T... a)` RETURNS `List<T>` O(1)	A **fixed-size** list backed by the array: `set` works, `add` and `remove` throw `UnsupportedOperationException`. Wrap it in `new ArrayList<>(...)` for a normal list. Passing an `int[]` gives a `List<int[]>` with one element, not a list of numbers.	`List<Integer> fixed = Arrays.asList(5, 3, 9);`
		`fixed.add(1);  // throws UnsupportedOperationException`
		`List<Integer> list = new ArrayList<>(Arrays.asList(5, 3, 9));`
		`list.add(1);  // list → [5, 3, 9, 1]`

`Arrays.stream(int[] a)` RETURNS `IntStream` O(n)	Stream over the array for one-line totals: `sum()`, `max()`, `min()`, `average()`. `sum()` of an `int[]` is an `int` and can overflow; use `Arrays.stream(a).asLongStream().sum()`.	`int sum = Arrays.stream(a).sum();  // → 18`
		`int max = Arrays.stream(a).max().getAsInt();  // → 9`
		`long big = Arrays.stream(new int[] {2_000_000_000, 2_000_000_000}).asLongStream().sum();  // → 4000000000`

ARRAYS & STRINGS String

27 entries

A `String` is **immutable**: every method that seems to change it returns a new string and leaves the original alone. Indexes are 0-based and every `end` index is exclusive.

IMPORT `java.lang (no import needed)`

START STATE · EVERY EXAMPLE BEGINS HERE

```
String s = "hello world";
```

READING CHARACTERS

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`length()` RETURNS `int` O(1)	Number of characters. A method with parentheses, unlike the array field `length`.	`int n = s.length();  // → 11`
`charAt(int index)` RETURNS `char` O(1)	Character at `index`. Throws `StringIndexOutOfBoundsException` outside 0..length−1.	`char c = s.charAt(4);  // → 'o'` `char bad = s.charAt(11);  // throws StringIndexOutOfBoundsException`
`isEmpty()` RETURNS `boolean` O(1)	True if `length() == 0`. A string of spaces is not empty.	`boolean e = "".isEmpty();  // → true` `boolean sp = " ".isEmpty();  // → false`
`isBlank()` JAVA 11+ RETURNS `boolean` O(n)	True if the string is empty or only whitespace.	`boolean b = "   ".isBlank();  // → true`

`toCharArray()` RETURNS `char[]` O(n)	A new `char[]` copy. Change it freely, then build a string again with `new String(chars)`.	`char[] cs = s.toCharArray();`
		`cs[0] = 'j';`
		`String t = new String(cs);  // → "jello world"`

`chars()` RETURNS `IntStream` O(n)	Stream of the characters as `int` values.	`long vowels = s.chars().filter(ch -> "aeiou".indexOf(ch) >= 0).count();  // → 3`

SEARCHING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`indexOf(String str)` RETURNS `int` `O(n·m)`	Index of the first occurrence, or −1. Overloads: `indexOf(char ch)`, and `indexOf(str, fromIndex)` to start the search later.	`int i = s.indexOf("o");  // → 4`
		`int next = s.indexOf("o", 5);  // → 7`
		`int none = s.indexOf('z');  // → -1`

`lastIndexOf(String str)` RETURNS `int` `O(n·m)`	Index of the last occurrence, or −1. Also takes a `char`.	`int j = s.lastIndexOf("o");  // → 7`
`contains(CharSequence s)` RETURNS `boolean` `O(n·m)`	True if the text occurs anywhere. Takes a `String` or `StringBuilder`, not a `char`: for a char use `indexOf(ch) >= 0`.	`boolean c = s.contains("lo w");  // → true`
`startsWith(String prefix)` RETURNS `boolean` `O(m)`	Also `endsWith(String suffix)`, and `startsWith(prefix, offset)`.	`boolean p = s.startsWith("he");  // → true` `boolean q = s.endsWith("ld");  // → true`

COMPARING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`equals(Object o)` RETURNS `boolean` `O(n)`	Same characters. Always use it for strings: `==` compares references and fails for strings built at runtime.	`String built = new StringBuilder("hel").append("lo").toString();`
		`boolean eq = built.equals("hello");  // → true`
		`boolean ref = built == "hello";  // → false`

`equalsIgnoreCase(String other)` RETURNS `boolean` `O(n)`	Equality ignoring letter case.	`boolean e = "Java".equalsIgnoreCase("JAVA");  // → true`

`compareTo(String other)` RETURNS `int` `O(n)`	Dictionary order: negative, 0 or positive. The value is the difference of the first differing characters, or of the lengths if one string is a prefix of the other. Upper case sorts before lower case. Also `compareToIgnoreCase`.	`int a = "apple".compareTo("banana");  // → -1`
		`int b = "apple".compareTo("app");  // → 2`
		`int c = "Zebra".compareTo("apple");  // → -7`

CUTTING AND CHANGING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`substring(int beginIndex, int endIndex)` RETURNS `String` `O(k)`	Characters beginIndex..endIndex−1 as a new string; `substring(beginIndex)` runs to the end. It copies, so it costs the length of the result.	`String w = s.substring(6);  // → "world"` `String h = s.substring(0, 5);  // → "hello"`

`split(String regex)` RETURNS `String[]` `O(n)`	Splits around matches of a **regular expression**, so `.` `|` `+` `*` `$` `?` need escaping: `split("\\.")`. Trailing empty strings are dropped; a leading empty string is kept.	`String[] words = "a  b c".split("\\s+");  // → [a, b, c]`
		`String[] parts = "1.2.3".split("\\.");  // → [1, 2, 3]`
		`String[] lead = ",a,,b,,".split(",");  // → [, a, , b]`

`split(String regex, int limit)` RETURNS `String[]` `O(n)`	With `limit` > 0, at most `limit` pieces (the last holds the rest). With a negative limit, trailing empty strings are kept.	`String[] kv = "key=a=b".split("=", 2);  // → [key, a=b]` `String[] all = "a,b,,".split(",", -1);  // → [a, b, , ]`
`trim()` RETURNS `String` `O(n)`	Removes leading and trailing whitespace. Java 11 adds `strip()`, which also handles Unicode spaces.	`String t = "  hi  ".trim();  // → "hi"`
`toLowerCase()` RETURNS `String` `O(n)`	Lower-case copy; `toUpperCase()` for upper case. Like every `String` method it returns a new string: `s.toUpperCase();` on its own changes nothing.	`String up = s.toUpperCase();  // → "HELLO WORLD"` `s.toUpperCase();  // s → "hello world"`
`replace(CharSequence target, CharSequence replacement)` RETURNS `String` `O(n)`	Replaces **every** occurrence, literally (no regex). `replace(char oldChar, char newChar)` for single characters.	`String r = s.replace("l", "L");  // → "heLLo worLd"` `String z = s.replace('o', '0');  // → "hell0 w0rld"`
`replaceAll(String regex, String replacement)` RETURNS `String` `O(n)`	Replaces every match of a regular expression.	`String digits = "a1b22c".replaceAll("[^0-9]", "");  // → "122"` `String letters = "A man, a plan".replaceAll("[^A-Za-z]", "").toLowerCase();  // → "amanaplan"`
`matches(String regex)` RETURNS `boolean` `O(n)`	True if the **whole** string matches the regex.	`boolean num = "12345".matches("\\d+");  // → true` `boolean part = "a1".matches("\\d+");  // → false`
`repeat(int count)` JAVA 11+ RETURNS `String` `O(n·count)`	The string `count` times. In Java 8, append in a loop with a `StringBuilder`.	`String line = "ab".repeat(3);  // → "ababab"`

`a + b` TAKES `a String and any value` RETURNS `String` `O(n + m)`	Concatenation; numbers and other values are converted to text. Evaluated left to right, so `1 + 2 + "x"` is `"3x"`. In a loop, `s += x` copies the whole string every time (O(n²) total): use `StringBuilder`.	`String t = "n=" + 5 + 3;  // → "n=53"`
		`String u = "n=" + (5 + 3);  // → "n=8"`
		`String v = 5 + 3 + "!";  // → "8!"`

BUILDING STRINGS FROM OTHER VALUES

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`String.valueOf(int i)` RETURNS `String` O(n)	Text of any primitive or object; overloads for `char`, `long`, `double`, `boolean`, `char[]` and `Object` (`null` gives `"null"`).	`String n = String.valueOf(42); // → "42"`
		`String c = String.valueOf('x'); // → "x"`
		`String hi = String.valueOf(new char[] {'h', 'i'}); // → "hi"`

`new String(char[] value)` RETURNS `String` O(n)	String from a `char` array (copied). `new String(chars, offset, count)` takes part of it. Sorting the letters gives an anagram key.	`char[] cs = "listen".toCharArray();`
		`Arrays.sort(cs);`
		`String key = new String(cs); // → "eilnst"`

`String.join(CharSequence delimiter, CharSequence... elements)` RETURNS `String` O(n)	Joins with a separator. Also accepts an `Iterable` such as a `List<String>`. Elements must be text, not numbers.	`String j = String.join("-", "a", "b", "c"); // → "a-b-c"`
		`List<String> parts = Arrays.asList("x", "y");`
		`String k = String.join(", ", parts); // → "x, y"`

`String.format(String format, Object... args)` RETURNS `String` O(n)	printf-style formatting: `%d` integer, `%s` text, `%.2f` two decimals, `%5d` width 5, `%05d` zero-padded, `%-3s` left-aligned, `%n` newline. Slow in big loops.	`String f = String.format("%.2f", 3.14159); // → "3.14"` `String p = String.format("%05d|%-3s|", 42, "ab"); // → "00042|ab |"`

PITFALL **Five String mistakes that cost test cases**

`==` instead of `equals`. `s += c` inside a loop (quadratic). Forgetting that `split` takes a regex. Treating the `end` of `substring` as inclusive. Comparing a `char` with a `String`: `s.charAt(0) == "a"` does not compile; write `'a'`.

ARRAYS & STRINGS

StringBuilder

14 entries

A mutable string. `append` at the end is O(1) amortised, so build answers and backtracking paths with it, then call `toString()` once.

IMPORT `java.lang (no import needed)`

START STATE · EVERY EXAMPLE BEGINS HERE

```
StringBuilder sb = new StringBuilder("abc");
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`new StringBuilder()` TAKES `nothing, a String, or an int capacity` RETURNS `StringBuilder` O(1)	Empty builder. `new StringBuilder("abc")` starts with text and `new StringBuilder(n)` reserves room for n characters. Trap: `new StringBuilder('a')` treats the char as a capacity (97) and starts **empty**.	`StringBuilder e = new StringBuilder();`
		`int len = e.length(); // → 0`
		`StringBuilder trap = new StringBuilder('a');`
		`String t = trap.toString(); // → ""`

`append(x)` TAKES `any type: String, char, int, long, double, boolean, char[], Object` RETURNS `StringBuilder` O(1) amortised	Adds `x` at the end and returns the same builder, so calls chain.	`sb.append('d').append(12).append(true); // sb → "abcd12true"`
`insert(int offset, x)` TAKES `int offset, then a value of any type` RETURNS `StringBuilder` O(n)	Inserts `x` before position `offset`. Shifts everything after it.	`sb.insert(0, "x"); // sb → "xabc"`
`deleteCharAt(int index)` RETURNS `StringBuilder` O(n - index)	Removes one character. Removing the last one, `sb.deleteCharAt(sb.length() - 1)`, is O(1): the usual undo step in backtracking.	`sb.deleteCharAt(sb.length() - 1); // sb → "ab"`
`delete(int start, int end)` RETURNS `StringBuilder` O(n)	Removes start..end−1.	`sb.delete(0, 2); // sb → "c"`

`setLength(int newLength)` RETURNS `void` O(1) when shrinking	Cuts the builder to `newLength` (or pads with `'\0'`). `setLength(0)` empties it for reuse; restoring a saved length undoes several appends at once.	`int mark = sb.length();`
		`sb.append("def");`
		`sb.setLength(mark); // sb → "abc"`

`reverse()` RETURNS `StringBuilder` O(n)	Reverses in place and returns the builder: the short way to reverse a string.	`String r = new StringBuilder("hello").reverse().toString(); // → "olleh"`
`charAt(int index)` RETURNS `char` O(1)	Character at `index`.	`char c = sb.charAt(1); // → 'b'`
`setCharAt(int index, char ch)` RETURNS `void` O(1)	Overwrites one character in place.	`sb.setCharAt(0, 'z'); // sb → "zbc"`
`length()` RETURNS `int` O(1)	Current number of characters.	`int n = sb.length(); // → 3`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`replace(int start, int end, String str)` RETURNS `StringBuilder` `O(n)`	Replaces start..end−1 with `str`.	`sb.replace(1, 2, "XYZ");  // sb → "aXYZc"`
`indexOf(String str)` RETURNS `int` `O(n·m)`	First index of `str`, or −1. Also `lastIndexOf`.	`int i = sb.indexOf("bc");  // → 1`
`substring(int start, int end)` RETURNS `String` `O(k)`	Copy of start..end−1 as a `String`.	`String t = sb.substring(1, 3);  // → "bc"`
`toString()` RETURNS `String` `O(n)`	The content as a `String`. Needed for `equals`, map keys and return values.	`String t = sb.toString();  // → "abc"`

PITFALL **StringBuilder has no content-based equals**

Two builders with the same text are not `equals`, and a builder never equals a `String`. Compare `toString()` results.

```
StringBuilder x = new StringBuilder("ab"), y = new StringBuilder("ab");
boolean wrong = x.equals(y);  // → false
boolean right = x.toString().equals(y.toString());  // → true
```

PART 3

Lists and the Collection basics

Collection<E> · ArrayList<E> and List<E> · LinkedList<E> · java.util.Collections · Iterator<E> and ListIterator<E>

LISTS Collection<E>: methods every list, set and queue has

14 entries

`ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, `ArrayDeque` and `PriorityQueue` all implement `Collection<E>`, so these calls work on each of them. The cost of `contains` and `remove` depends on the class: O(1) for `HashSet`, O(log n) for `TreeSet`, O(n) for lists, deques and priority queues.

IMPORT `java.util.*`

START STATE · EVERY EXAMPLE BEGINS HERE

```
Collection<Integer> c = new ArrayList<>(Arrays.asList(1, 2, 3, 2));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`size()` RETURNS `int` `O(1)`	Number of elements.	`int n = c.size();  // → 4`
`isEmpty()` RETURNS `boolean` `O(1)`	Same as `size() == 0`.	`boolean e = c.isEmpty();  // → false`
`contains(Object o)` RETURNS `boolean` `O(1) to O(n)`	Membership test using `equals`.	`boolean has = c.contains(3);  // → true`
`add(E e)` RETURNS `boolean` `O(1)`	Adds `e` and returns true if the collection changed: always for lists, false for a set that already had `e`.	`boolean changed = c.add(5);  // → true`
`remove(Object o)` RETURNS `boolean` `O(1) to O(n)`	Removes **one** element equal to `o`; true if one was found.	`boolean r = c.remove(Integer.valueOf(2));  // → true` `// c → [1, 3, 2]`
`addAll(Collection<? extends E> c)` RETURNS `boolean` `O(m)`	Adds every element of another collection. For sets this is the union.	`c.addAll(Arrays.asList(7, 8));  // c → [1, 2, 3, 2, 7, 8]`
`removeAll(Collection<?> c)` RETURNS `boolean` `O(n·cost of contains)`	Removes every element that also appears in the argument, all copies.	`c.removeAll(Arrays.asList(2));  // c → [1, 3]`
`retainAll(Collection<?> c)` RETURNS `boolean` `O(n·cost of contains)`	Keeps only elements that also appear in the argument. For sets this is the intersection.	`c.retainAll(Arrays.asList(2, 3));  // c → [2, 3, 2]`
`containsAll(Collection<?> c)` RETURNS `boolean` `O(m·cost of contains)`	True if every element of the argument is present.	`boolean all = c.containsAll(Arrays.asList(1, 3));  // → true`
`removeIf(Predicate<? super E> filter)` RETURNS `boolean` `O(n)`	Removes every element that matches the condition. The safe way to delete while scanning.	`c.removeIf(x -> x % 2 == 0);  // c → [1, 3]`
`clear()` RETURNS `void` `O(n)`	Removes everything.	`c.clear();  // c → []`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`forEach(Consumer<? super E> action)` RETURNS `void` O(n)	Runs `action` on each element in iteration order.	`StringBuilder out = new StringBuilder(); c.forEach(x -> out.append(x)); // out → "1232"`
`toArray(T[] a)` RETURNS `T[]` O(n)	Copies into an object array: `toArray(new Integer[0])` returns an `Integer[]`. There is no direct way to get an `int[]`: use `stream().mapToInt(Integer::intValue).toArray()`.	`Integer[] boxed = c.toArray(new Integer[0]); // → [1, 2, 3, 2]` `int[] prim = c.stream().mapToInt(Integer::intValue).toArray(); // → [1, 2, 3, 2]`
`stream()` RETURNS `Stream<E>` O(1) to create	A stream over the elements; see the Streams section.	`int sum = c.stream().mapToInt(Integer::intValue).sum(); // → 8`

LISTS ArrayList<E> and List<E>

18 entries

A resizable array: O(1) reads and writes by index, O(1) amortised `add` at the end, O(n) inserts and removals elsewhere. Declare the variable as `List<Integer>` and create an `ArrayList`. Everything from the Collection section also applies.

IMPORT `java.util.ArrayList, java.util.List`

START STATE · EVERY EXAMPLE BEGINS HERE

```
List<Integer> list = new ArrayList<>(Arrays.asList(5, 3, 7));
```

CREATING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`new ArrayList<>()` TAKES `nothing, an int capacity, or a Collection to copy` RETURNS `ArrayList<E>` O(1), copy O(n)	Empty list. `new ArrayList<>(n)` reserves room for n elements; `new ArrayList<>(other)` copies another collection, the way to save a snapshot of a backtracking path.	`List<Integer> empty = new ArrayList<>(); List<Integer> copy = new ArrayList<>(list); copy.add(1); // list → [5, 3, 7]`
`List.of(E... elements)` JAVA 9+ RETURNS `List<E>` O(n)	Immutable list: every change throws `UnsupportedOperationException`, and `null` elements are rejected. In Java 8 use `Arrays.asList` (fixed size) or `new ArrayList<>(Arrays.asList(...))`.	`List<Integer> fixed = List.of(1, 2, 3); fixed.set(0, 9); // throws UnsupportedOperationException`

READING AND WRITING BY INDEX

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`get(int index)` RETURNS `E` O(1)	Element at `index`. Throws `IndexOutOfBoundsException` outside 0..size−1.	`int x = list.get(0); // → 5` `int bad = list.get(3); // throws IndexOutOfBoundsException`
`set(int index, E element)` RETURNS `E` O(1)	Replaces the element at `index` and returns the **old** one.	`int old = list.set(1, 10); // → 3` `// list → [5, 10, 7]`
`size()` RETURNS `int` O(1)	Number of elements; lists have `size()`, not `length`.	`int n = list.size(); // → 3`

ADDING AND REMOVING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`add(E e)` RETURNS `boolean` O(1) amortised	Appends at the end. Always returns true.	`list.add(9); // list → [5, 3, 7, 9]`
`add(int index, E element)` RETURNS `void` O(n - index)	Inserts at `index`, shifting later elements right. `index` may equal `size()`, which appends.	`list.add(0, 1); // list → [1, 5, 3, 7]`
`remove(int index)` RETURNS `E` O(n - index)	Removes by **index** and returns the removed element. Removing the last element is O(1).	`int gone = list.remove(0); // → 5` `// list → [3, 7]` `list.remove(list.size() - 1); // list → [3]`
`remove(Object o)` RETURNS `boolean` O(n)	Removes the first element equal to `o`; true if found. On a `List<Integer>`, `remove(7)` means **index** 7, so box the value: `remove(Integer.valueOf(7))`.	`boolean found = list.remove(Integer.valueOf(7)); // → true` `// list → [5, 3]`
`addAll(Collection<? extends E> c)` RETURNS `boolean` O(m)	Appends every element; `addAll(index, c)` inserts them at a position.	`list.addAll(Arrays.asList(1, 2)); // list → [5, 3, 7, 1, 2]`
`clear()` RETURNS `void` O(n)	Removes everything.	`list.clear(); // list → []`

SEARCHING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`indexOf(Object o)` RETURNS `int` O(n)	First index of `o`, or −1. Also `lastIndexOf`.	`int i = list.indexOf(7); // → 2` `int none = list.indexOf(4); // → -1`
`contains(Object o)` RETURNS `boolean` O(n)	Linear search. Calling it inside a loop makes the loop O(n²); keep a `HashSet` next to the list if you test membership often.	`boolean has = list.contains(3); // → true`

ORDERING AND VIEWS

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`sort(Comparator<? super E> c)` RETURNS `void` O(n log n)	Sorts in place (stable). Pass `null` for natural order and `Comparator.reverseOrder()` for descending. Same as `Collections.sort`.	`list.sort(null); // list → [3, 5, 7]`
		`list.sort(Comparator.reverseOrder()); // list → [7, 5, 3]`

`subList(int fromIndex, int toIndex)` RETURNS `List<E>` O(1)	A **view** of fromIndex..toIndex−1: changes to it change the original list. `list.subList(a, b).clear()` deletes a range.	`List<Integer> v = list.subList(0, 2); // → [5, 3]` `v.clear(); // list → [7]`
`replaceAll(UnaryOperator<E> operator)` RETURNS `void` O(n)	Replaces each element with `operator(element)`.	`list.replaceAll(x -> x * 2); // list → [10, 6, 14]`
`equals(Object o)` RETURNS `boolean` O(n)	True if the other list has equal elements in the same order. Lists also have a matching `hashCode`, which is why a `List<Integer>` works as a `HashMap` key.	`boolean eq = list.equals(Arrays.asList(5, 3, 7)); // → true`
`getLast()` JAVA 21+ RETURNS `E` O(1)	Last element; `getFirst()`, `removeFirst()`, `removeLast()` and `reversed()` also arrived with Java 21's `SequencedCollection`. Throws `NoSuchElementException` when empty. In Java 8 write `list.get(list.size() - 1)`.	`int last = list.getLast(); // → 7` `int java8 = list.get(list.size() - 1); // → 7`

PITFALL **Saving a backtracking path**

`result.add(path)` stores a reference to the same list you keep changing. Store a copy.

```
List<List<Integer>> result = new ArrayList<>();
List<Integer> path = new ArrayList<>(Arrays.asList(1, 2));
result.add(path);                  // reference: changes later
result.add(new ArrayList<>(path));   // snapshot: stays [1, 2]
path.add(3);
// result → [[1, 2, 3], [1, 2]]
```

PITFALL **Comparing two elements of a List<Integer> with ==**

`list.get(i) == list.get(j)` compares two `Integer` objects by reference. It works for −128..127 because of the cache and fails above. Use `equals`, or unbox into `int` variables first.

```
List<Integer> xs = new ArrayList<>(Arrays.asList(1000, 1000));
boolean ref = xs.get(0) == xs.get(1);  // → false
boolean val = xs.get(0).equals(xs.get(1));  // → true
int p = xs.get(0), q = xs.get(1);
boolean prim = p == q;  // → true
```

A grid stored as List<List<Integer>> TEMPLATE

```
List<List<Integer>> grid = new ArrayList<>();
for (int r = 0; r < 2; r++) {
    grid.add(new ArrayList<>());
    for (int c = 0; c < 3; c++) grid.get(r).add(r * 3 + c);
}
int cell = grid.get(1).get(2);  // → 5
grid.get(0).set(0, 9);
// grid → [[9, 1, 2], [3, 4, 5]]
```

LISTS LinkedList<E>

5 entries

A doubly linked list that implements both `List` and `Deque`. Ends are O(1); anything by index is O(n). `ArrayDeque` is faster for stacks and queues; reach for `LinkedList` mainly when you need to store `null` in a queue.

IMPORT `java.util.LinkedList`

START STATE · EVERY EXAMPLE BEGINS HERE

```
LinkedList<Integer> ll = new LinkedList<>(Arrays.asList(1, 2, 3));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`addFirst(E e)` RETURNS `void` O(1)	Adds at the front; `addLast` at the back.	`ll.addFirst(0); // ll → [0, 1, 2, 3]`
`removeFirst()` RETURNS `E` O(1)	Removes and returns the front; `removeLast` for the back. Both throw `NoSuchElementException` when empty; `pollFirst` and `pollLast` return `null` instead.	`int f = ll.removeFirst(); // → 1` `int l = ll.removeLast(); // → 3`
`getFirst()` RETURNS `E` O(1)	Front element without removing; `getLast` for the back. `peekFirst` / `peekLast` return `null` when empty.	`int f = ll.getFirst(); // → 1`
`get(int index)` RETURNS `E` O(n)	Walks from the nearer end, so a loop of `get(i)` over a `LinkedList` is O(n²). Iterate instead.	`int mid = ll.get(1); // → 2`
`offer(E e)` RETURNS `boolean` O(1)	All queue and stack methods work (`offer`, `poll`, `peek`, `push`, `pop`). Unlike `ArrayDeque`, it accepts `null`.	`ll.offer(null);` `int n = ll.size(); // → 4`

LISTS java.util.Collections

13 entries

Static helpers for lists and other collections.

IMPORT `java.util.Collections`

START STATE · EVERY EXAMPLE BEGINS HERE

```
List<Integer> list = new ArrayList<>(Arrays.asList(4, 1, 3, 1));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Collections.sort(List<T> list)` RETURNS `void` O(n log n)	Sorts ascending in place; stable (TimSort). `Collections.sort(list, comparator)` for any other order.	`Collections.sort(list); // list → [1, 1, 3, 4]`

`Collections.reverseOrder()` RETURNS `Comparator<T>` O(1)	Comparator for descending natural order; `reverseOrder(cmp)` reverses another comparator. Also the usual way to build a max-heap.	`Collections.sort(list, Collections.reverseOrder()); // list → [4, 3, 1, 1]`
		`PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());`

`Collections.reverse(List<?> list)` RETURNS `void` O(n)	Reverses in place.	`Collections.reverse(list); // list → [1, 3, 1, 4]`
`Collections.max(Collection<? extends T> coll)` RETURNS `T` O(n)	Largest element; `Collections.min` for the smallest; both have a version that takes a comparator. They throw `NoSuchElementException` on an empty collection.	`int mx = Collections.max(list); // → 4` `int mn = Collections.min(list); // → 1`
`Collections.swap(List<?> list, int i, int j)` RETURNS `void` O(1)	Swaps the elements at two positions.	`Collections.swap(list, 0, 1); // list → [1, 4, 3, 1]`
`Collections.frequency(Collection<?> c, Object o)` RETURNS `int` O(n)	How many elements equal `o`.	`int ones = Collections.frequency(list, 1); // → 2`
`Collections.nCopies(int n, T o)` RETURNS `List<T>` O(1)	Immutable list of `n` copies of `o`; wrap it in `new ArrayList<>(...)` to edit.	`List<Integer> zeros = new ArrayList<>(Collections.nCopies(3, 0)); // → [0, 0, 0]`
`Collections.addAll(Collection<? super T> c, T... elements)` RETURNS `boolean` O(k)	Adds several values in one call.	`Collections.addAll(list, 9, 8); // list → [4, 1, 3, 1, 9, 8]`
`Collections.binarySearch(List<? extends T> list, T key)` RETURNS `int` O(log n)	Same contract as `Arrays.binarySearch`, on a sorted list.	`List<Integer> sorted = Arrays.asList(1, 3, 5);` `int i = Collections.binarySearch(sorted, 3); // → 1`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`Collections.emptyList()` RETURNS `List<T>` O(1)	Immutable empty list; also `emptySet()` and `emptyMap()`.	`List<Integer> none = Collections.emptyList(); // → []`
`Collections.unmodifiableList(List<? extends T> list)` RETURNS `List<T>` O(1)	Read-only view: writes through it throw `UnsupportedOperationException`.	`List<Integer> ro = Collections.unmodifiableList(list); ro.add(5); // throws UnsupportedOperationException`
`Collections.shuffle(List<?> list, Random rnd)` RETURNS `void` O(n)	Random order; `shuffle(list)` without a seed.	`Collections.shuffle(list, new Random(1)); int n = list.size(); // → 4`
`Collections.fill(List<? super T> list, T obj)` RETURNS `void` O(n)	Overwrites every element with `obj`.	`Collections.fill(list, 0); // list → [0, 0, 0, 0]`

LISTS Iterator<E> and ListIterator<E>

5 entries

A for-each loop uses an iterator behind the scenes. Use one explicitly when you need to delete while looping.

IMPORT `java.util.Iterator, java.util.ListIterator`

START STATE · EVERY EXAMPLE BEGINS HERE

```
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`iterator()` RETURNS `Iterator<E>` O(1)	A cursor over any collection (also maps' `keySet()`, `values()`, `entrySet()`).	`Iterator<Integer> it = list.iterator(); int first = it.next(); // → 1`
`hasNext()` RETURNS `boolean` O(1)	True if `next()` has an element to return.	`Iterator<Integer> it = list.iterator(); boolean more = it.hasNext(); // → true`
`next()` RETURNS `E` O(1)	Returns the next element and moves past it. Throws `NoSuchElementException` after the last one.	`Iterator<Integer> it = new ArrayList<Integer>().iterator(); it.next(); // throws NoSuchElementException`
`remove()` RETURNS `void` O(n) for ArrayList, O(1) for others	Removes the element that the last `next()` returned. Together with `removeIf`, the only safe way to delete during a loop.	`Iterator<Integer> it = list.iterator(); while (it.hasNext()) if (it.next() % 2 == 0) it.remove(); // list → [1, 3]`
`listIterator()` RETURNS `ListIterator<E>` O(1)	List iterator that can also move backwards (`hasPrevious`, `previous`) and edit at the cursor (`set`, `add`).	`ListIterator<Integer> it = list.listIterator(); while (it.hasNext()) it.set(it.next() * 10); // list → [10, 20, 30, 40]`

PITFALL Removing inside a for-each loop

Changing a collection while a for-each loop walks it throws `ConcurrentModificationException`. Use `removeIf`, an explicit `Iterator`, or collect the changes and apply them after the loop.

```
for (int x : list) if (x == 2) list.remove(Integer.valueOf(x));  // throws ConcurrentModificationException
```

PART 4

Maps and sets

HashMap<K, V> and Map<K, V> | LinkedHashMap and LinkedHashSet | TreeMap<K, V> | HashSet<E> and Set<E> | TreeSet<E>

MAPS & SETS

HashMap<K, V> and Map<K, V>

26 entries

Key → value lookup in O(1) expected time. Declare as `Map<K, V>` and create a `HashMap`. Iteration order is **not** defined: never rely on it, and use `TreeMap` or `LinkedHashMap` when order matters. (The printed maps below happen to come out in a fixed order because the keys are small.)

IMPORT `java.util.HashMap, java.util.Map`

START STATE · EVERY EXAMPLE BEGINS HERE

```
Map<String, Integer> map = new HashMap<>();
map.put("a", 1);
map.put("b", 2);
```

CREATING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`new HashMap<>()`
TAKES `nothing, an int capacity, or a Map to copy`
RETURNS `HashMap<K, V>`
`O(1), copy O(n)`
	Empty map; `new HashMap<>(other)` copies a map. Keys need working `equals` and `hashCode`: `String`, `Integer`, `Long`, `Character` and `List` have them, arrays do not.	`Map<Integer, List<Integer>> graph = new HashMap<>();`
		`Map<String, Integer> copy = new HashMap<>(map);  // → {a=1, b=2}`

`Map.of(K k1, V v1, ...)` JAVA 9+ RETURNS `Map<K, V>` `O(n)`	Immutable map of up to 10 key–value pairs.	`Map<String, Integer> m = Map.of("x", 1, "y", 2);` `int v = m.get("x");  // → 1`

READING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`get(Object key)`
RETURNS `V` `O(1)`
	Value for `key`, or `null` if the key is absent. Assigning that `null` to an `int` throws `NullPointerException`.	`Integer v = map.get("a");  // → 1`
		`Integer none = map.get("z");  // → null`
		`int boom = map.get("z");  // throws NullPointerException`

`getOrDefault(Object key, V defaultValue)` RETURNS `V` `O(1)`	Value for `key`, or `defaultValue` if absent. It does **not** insert the default.	`int c = map.getOrDefault("z", 0);  // → 0` `boolean inserted = map.containsKey("z");  // → false`
`containsKey(Object key)` RETURNS `boolean` `O(1)`	True if the key is present.	`boolean has = map.containsKey("b");  // → true`
`containsValue(Object value)` RETURNS `boolean` `O(n)`	Scans every entry.	`boolean has = map.containsValue(2);  // → true`
`size()` RETURNS `int` `O(1)`	Number of keys; also `isEmpty()`.	`int n = map.size();  // → 2`

WRITING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`put(K key, V value)`
RETURNS `V` `O(1)`
	Inserts or overwrites. Returns the **previous** value, or `null` if the key was new.	`Integer prev = map.put("a", 10);  // → 1`
		`Integer fresh = map.put("c", 3);  // → null`
		`// map → {a=10, b=2, c=3}`

`putIfAbsent(K key, V value)` RETURNS `V` `O(1)`	Inserts only if the key is absent (or mapped to `null`). Returns the existing value, or `null` if it inserted.	`Integer existing = map.putIfAbsent("a", 99);  // → 1` `map.putIfAbsent("d", 4);  // map → {a=1, b=2, d=4}`

`merge(K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)`
RETURNS `V` `O(1)`
	If `key` is absent, stores `value`; otherwise stores `f(oldValue, value)`. Returns the new value. If `f` returns `null`, the key is removed. `merge(k, 1, Integer::sum)` is the one-line frequency counter.	`map.merge("a", 1, Integer::sum);  // map → {a=2, b=2}`
		`map.merge("z", 1, Integer::sum);  // map → {a=2, b=2, z=1}`

`computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction)`
RETURNS `V` `O(1)`
	Returns the value for `key`, first creating it with `f(key)` if absent. The one-line way to group values into lists.	`Map<Integer, List<String>> byLen = new HashMap<>();`
		`byLen.computeIfAbsent(3, k -> new ArrayList<>()).add("cat");`
		`byLen.computeIfAbsent(3, k -> new ArrayList<>()).add("dog");`
		`// byLen → {3=[cat, dog]}`

`compute(K key, BiFunction<? super K, ? super V, ? extends V> remappingFunction)` RETURNS `V` `O(1)`	Stores `f(key, oldValueOrNull)` and returns it; returning `null` removes the key.	`map.compute("a", (k, v) -> v == null ? 1 : v * 10);  // map → {a=10, b=2}`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`computeIfPresent(K key, BiFunction<? super K, ? super V, ? extends V> remappingFunction)` RETURNS `V` O(1)	Runs only if the key is present; returning `null` removes the key. Handy for "decrement, and delete at zero".	`map.computeIfPresent("a", (k, v) -> v - 1 == 0 ? null : v - 1); // map → {b=2}`
`remove(Object key)` RETURNS `V` O(1)	Deletes the key and returns its old value, or `null`. `remove(key, value)` deletes only if the key maps to that value.	`Integer old = map.remove("a"); // → 1` `// map → {b=2}`
`replace(K key, V value)` RETURNS `V` O(1)	Overwrites only if the key already exists; returns the old value or `null`.	`Integer r = map.replace("z", 5); // → null` `// map → {a=1, b=2}`
`putAll(Map<? extends K, ? extends V> m)` RETURNS `void` O(m)	Copies every pair from another map, overwriting equal keys.	`map.putAll(Collections.singletonMap("b", 20)); // map → {a=1, b=20}`
`clear()` RETURNS `void` O(capacity)	Removes every key.	`map.clear(); // map → {}`

LOOPING OVER A MAP

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`keySet()` RETURNS `Set<K>` O(1) to create	A live view of the keys. Removing a key from it removes it from the map.	`Set<String> keys = map.keySet(); // → [a, b]`

`values()` RETURNS `Collection<V>` O(1) to create	A live view of the values.	`int total = 0;`
		`for (int v : map.values()) total += v;`
		`// total → 3`

`entrySet()` RETURNS `Set<Map.Entry<K, V>>` O(1) to create	A live view of the key–value pairs: the fastest way to loop over both.	`StringBuilder sb = new StringBuilder();`
		`for (Map.Entry<String, Integer> e : map.entrySet())`
		`  sb.append(e.getKey()).append(e.getValue());`
		`// sb → "a1b2"`

`forEach(BiConsumer<? super K, ? super V> action)` RETURNS `void` O(n)	Runs `action(key, value)` for every pair.	`StringBuilder sb = new StringBuilder();`
		`map.forEach((k, v) -> sb.append(k).append('=').`
		`append(v).append(' '));`
		`// sb → "a=1 b=2 "`

MAP.ENTRY<K, V>: ONE KEY–VALUE PAIR

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`getKey()` RETURNS `K` O(1)	Key of an entry from `entrySet()`, `TreeMap.firstEntry()` and similar.	`Map.Entry<String, Integer> e =`
		`map.entrySet().iterator().next();`
		`String k = e.getKey(); // → "a"`

`getValue()` RETURNS `V` O(1)	Value of the entry.	`Map.Entry<String, Integer> e =`
		`map.entrySet().iterator().next();`
		`int v = e.getValue(); // → 1`

`setValue(V value)` RETURNS `V` O(1)	Changes the value inside the map while you loop over `entrySet()`. Entries returned by `TreeMap` methods such as `firstEntry()` are read-only snapshots and throw `UnsupportedOperationException`.	`for (Map.Entry<String,`
		`Integer> e : map.entrySet())`
		`e.setValue(e.getValue() * 10);`
		`// map → {a=10, b=20}`

`new AbstractMap.SimpleEntry<>(K key, V value)` RETURNS `Map.Entry<K, V>` O(1)	A standalone pair object that works in Java 8. Java 9 adds `Map.entry(k, v)`, which is immutable.	`Map.Entry<String, Integer> p = new`
		`AbstractMap.SimpleEntry<>("x", 5);`
		`int v = p.getValue(); // → 5`

`Map.Entry.comparingByValue()` RETURNS `Comparator<Map.Entry<K, V>>` O(1)	Comparator over entries by value; `comparingByKey()` by key. Both accept a comparator argument, e.g. `Comparator.reverseOrder()`. Used to sort entries, as in "top k frequent elements".	`List<Map.Entry<String, Integer>> es = new`
		`ArrayList<>(map.entrySet());`
		`es.sort(Map.Entry.comparingByValue(Comparator.`
		`reverseOrder()));`
		`String top = es.get(0).getKey(); // → "b"`

Counting: HashMap or int[26]

TEMPLATE

```
String text = "banana";
Map<Character, Integer> freq = new HashMap<>();
for (char ch : text.toCharArray()) freq.merge(ch, 1, Integer::sum);
// freq → {a=3, b=1, n=2}

int[] cnt = new int[26];                          // faster when keys are 'a'..'z'
for (char ch : text.toCharArray()) cnt[ch - 'a']++;
int as = cnt['a' - 'a'];  // → 3

if (freq.merge('n', -1, Integer::sum) == 0) freq.remove('n');   // remove one 'n'
freq.values().removeIf(v -> v == 0);                             // drop all zero counts
// freq → {a=3, b=1, n=1}
```

PITFALL **Arrays as keys, and keys for pairs**

An `int[]` key uses identity, so two equal arrays are two different keys. Convert the key: a `List<Integer>`, a `String` such as `r + "," + c`, or one `long` / `int` that encodes both numbers.

```
Map<int[], Integer> bad = new HashMap<>();
bad.put(new int[] {1, 2}, 1);
boolean found = bad.containsKey(new int[] {1, 2});  // → false

Map<List<Integer>, Integer> byList = new HashMap<>();
byList.put(Arrays.asList(1, 2), 1);
boolean ok = byList.containsKey(Arrays.asList(1, 2));  // → true

int cols = 1000;
Set<Integer> cells = new HashSet<>();
cells.add(3 * cols + 7);                          // cell (3, 7) as one int
boolean seen = cells.contains(3 * cols + 7);  // → true
long pairKey = ((long) 5 << 32) | 9;              // two ints packed into one long
```

Your own class as a key: override equals and hashCode

TEMPLATE

```
class Point {
    final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
    @Override public boolean equals(Object o) {
        if (!(o instanceof Point)) return false;
        Point p = (Point) o;
        return x == p.x && y == p.y;
    }
    @Override public int hashCode() { return 31 * x + y; }
}

Set<Point> seen = new HashSet<>();
seen.add(new Point(1, 2));
boolean found = seen.contains(new Point(1, 2));  // → true
```

MAPS & SETS

LinkedHashMap and LinkedHashSet

5 entries

A `HashMap` or `HashSet` that also remembers order: insertion order by default, or access order for an LRU cache. Same O(1) costs.

IMPORT `java.util.LinkedHashMap, java.util.LinkedHashSet`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`new LinkedHashMap<>()`
TAKES `nothing, or a Map to copy`
RETURNS `LinkedHashMap<K, V>` `O(1)`
	Iterates in **insertion order**. Overwriting an existing key keeps its original position.	`Map<String, Integer> m = new LinkedHashMap<>();`
		`m.put("z", 1);`
		`m.put("a", 2);`
		`m.put("z", 3);`
		`// m → {z=3, a=2}`

`new LinkedHashMap<>(int initialCapacity, float loadFactor, boolean accessOrder)`
RETURNS `LinkedHashMap<K, V>` `O(1)`
	With `accessOrder = true`, every `get` or `put` moves that key to the end, so the first key is the least recently used.	`LinkedHashMap<Integer, Integer> m = new LinkedHashMap<>(16, 0.75f, true);`
		`m.put(1, 1);`
		`m.put(2, 2);`
		`m.get(1);`
		`// m → {2=2, 1=1}`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`removeEldestEntry(Map.Entry<K, V> eldest)` RETURNS `boolean` O(1)	Protected method called after every `put`. Override it to return true and the eldest entry is dropped. With access order this is a complete LRU cache (LeetCode 146).	`final int capacity = 2;`
		`LinkedHashMap<Integer, Integer> lru = new LinkedHashMap<Integer, Integer>(16, 0.75f, true) {`
		`  protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {`
		`  return size() > capacity;`
		`  }`
		`};`
		`lru.put(1, 1);`
		`lru.put(2, 2);`
		`lru.get(1);  // 1 is now the most recently used`
		`lru.put(3, 3);  // evicts 2`
		`// lru → {1=1, 3=3}`

`new LinkedHashSet<>()` TAKES `nothing, or a Collection to copy` RETURNS `LinkedHashSet<E>` O(1)	Set that iterates in insertion order: removes duplicates and keeps the first-seen order.	`Set<Integer> s = new LinkedHashSet<>(Arrays.asList(3, 1, 3, 2, 1));` `// s → [3, 1, 2]`

`pollFirstEntry()` JAVA 21+ RETURNS `Map.Entry<K, V>` O(1)	Java 21 adds `firstEntry`, `lastEntry`, `pollFirstEntry`, `pollLastEntry`, `putFirst` and `putLast` to `LinkedHashMap`. In Java 8, read the oldest key with `map.keySet().iterator().next()`.	`LinkedHashMap<String, Integer> m = new LinkedHashMap<>();`
		`m.put("x", 1);`
		`m.put("y", 2);`
		`String oldest = m.pollFirstEntry().getKey();  // → "x"`
		`String java8 = m.keySet().iterator().next();  // → "y"`

MAPS & SETS

TreeMap<K, V>: a sorted map

15 entries

A red-black tree: keys are kept sorted, and every `Map` method costs O(log n). On top of `Map` it answers “nearest key below or above x” and “smallest/largest key”. Declare the variable as `TreeMap` (not `Map`) so these methods are visible.

IMPORT `java.util.TreeMap`

START STATE · EVERY EXAMPLE BEGINS HERE

```
TreeMap<Integer, String> tm = new TreeMap<>();
tm.put(10, "a");
tm.put(20, "b");
tm.put(30, "c");
```

CREATING AND THE USUAL MAP CALLS

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`new TreeMap<>()` TAKES `nothing, a Comparator<? super K>, or a Map to copy` RETURNS `TreeMap<K, V>` O(1)	Keys in natural order; `new TreeMap<>(Comparator.reverseOrder())` for descending. Keys must be comparable and not `null`.	`TreeMap<String, Integer> desc = new TreeMap<>(Comparator.reverseOrder());`
		`desc.put("a", 1);`
		`desc.put("c", 3);`
		`desc.put("b", 2);`
		`// desc → {c=3, b=2, a=1}`

`put(K key, V value)` RETURNS `V` O(log n)	Same behaviour as `HashMap` (as are `get`, `getOrDefault`, `merge`, `remove`, `containsKey`), but O(log n), and iteration is in key order.	`tm.put(15, "x");` `// tm → {10=a, 15=x, 20=b, 30=c}`
`keySet()` RETURNS `Set<K>` O(1) to create	Keys in ascending order; `values()` and `entrySet()` follow the same order.	`Set<Integer> ks = tm.keySet();  // → [10, 20, 30]`

SMALLEST AND LARGEST

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`firstKey()` RETURNS `K` O(log n)	Smallest key; `lastKey()` for the largest. Both throw `NoSuchElementException` on an empty map.	`int lo = tm.firstKey();  // → 10`
		`int hi = tm.lastKey();  // → 30`
		`new TreeMap<Integer, String>().firstKey();  // throws NoSuchElementException`

`firstEntry()` RETURNS `Map.Entry<K, V>` O(log n)	Smallest key with its value, or `null` if empty; `lastEntry()` for the largest.	`Map.Entry<Integer, String> e = tm.firstEntry();  // → 10=a` `Map.Entry<Integer, String> none = new TreeMap<Integer, String>().firstEntry();  // → null`
`pollFirstEntry()` RETURNS `Map.Entry<K, V>` O(log n)	Removes and returns the smallest entry, or `null` if empty; `pollLastEntry()` for the largest.	`Map.Entry<Integer, String> e = tm.pollFirstEntry();  // → 10=a` `// tm → {20=b, 30=c}`

NEAREST KEY

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`floorKey(K key)` RETURNS `K` `O(log n)`	Greatest key ≤ `key`, or `null` if there is none.	`Integer f = tm.floorKey(25); // → 20`
		`Integer g = tm.floorKey(20); // → 20`
		`Integer none = tm.floorKey(5); // → null`

`ceilingKey(K key)` RETURNS `K` `O(log n)`	Least key ≥ `key`, or `null`.	`Integer c = tm.ceilingKey(25); // → 30` `Integer none = tm.ceilingKey(31); // → null`
`lowerKey(K key)` RETURNS `K` `O(log n)`	Greatest key **strictly** < `key`, or `null`.	`Integer l = tm.lowerKey(20); // → 10`
`higherKey(K key)` RETURNS `K` `O(log n)`	Least key **strictly** > `key`, or `null`.	`Integer h = tm.higherKey(20); // → 30`
`floorEntry(K key)` RETURNS `Map.Entry<K, V>` `O(log n)`	Like `floorKey` but returns the key–value pair (or `null`). Also `ceilingEntry`, `lowerEntry`, `higherEntry`.	`Map.Entry<Integer, String> e = tm.floorEntry(25); // → 20=b` `String v = e.getValue(); // → "b"`

RANGES AND REVERSE ORDER

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`headMap(K toKey)` RETURNS `SortedMap<K, V>` `O(log n) to create`	View of the keys < `toKey`. `headMap(toKey, true)` includes `toKey` and returns a `NavigableMap`.	`SortedMap<Integer, String> h = tm.headMap(20); // → {10=a}` `NavigableMap<Integer, String> hi = tm.headMap(20, true); // → {10=a, 20=b}`
`tailMap(K fromKey)` RETURNS `SortedMap<K, V>` `O(log n) to create`	View of the keys ≥ `fromKey`; `tailMap(fromKey, false)` excludes it.	`SortedMap<Integer, String> t = tm.tailMap(20); // → {20=b, 30=c}`
`subMap(K fromKey, K toKey)` RETURNS `SortedMap<K, V>` `O(log n) to create`	View of the keys in [fromKey, toKey); `subMap(from, fromInclusive, to, toInclusive)` for other bounds. Views write through to the map.	`SortedMap<Integer, String> s = tm.subMap(10, 30); // → {10=a, 20=b}` `tm.subMap(10, true, 20, true).clear(); // tm → {30=c}`
`descendingMap()` RETURNS `NavigableMap<K, V>` `O(1)`	Reverse-order view; `descendingKeySet()` for just the keys.	`NavigableMap<Integer, String> d = tm.descendingMap(); // → {30=c, 20=b, 10=a}`

PITFALL **Nearest-key methods return null**

`floorKey`, `ceilingKey`, `lowerKey` and `higherKey` return `Integer`, which is `null` when nothing qualifies. Unboxing it straight into an `int` throws.

```
Integer f = tm.floorKey(5);
int safe = f == null ? -1 : f;  // → -1
int boom = tm.floorKey(5);  // throws NullPointerException
```

A multiset (sorted bag with duplicates) on TreeMap

TEMPLATE

```
TreeMap<Integer, Integer> bag = new TreeMap<>();
for (int x : new int[] {5, 1, 5, 3}) bag.merge(x, 1, Integer::sum);       // add
// bag → {1=1, 3=1, 5=2}
if (bag.merge(5, -1, Integer::sum) == 0) bag.remove(5);                  // remove one 5
// bag → {1=1, 3=1, 5=1}
int smallest = bag.firstKey();  // → 1
int largest = bag.lastKey();  // → 5
```

Java has no built-in multiset. Count copies in a `TreeMap`; this is also how to get a "heap with delete" in O(log n), e.g. for a sliding-window median.

MAPS & SETS

HashSet<E> and Set<E>

11 entries

Unique elements with O(1) expected `add`, `remove` and `contains`. Order is not defined; use `TreeSet` for sorted order or `LinkedHashSet` for insertion order.

IMPORT `java.util.HashSet, java.util.Set`

START STATE · EVERY EXAMPLE BEGINS HERE

```
Set<Integer> set = new HashSet<>(Arrays.asList(1, 2, 3));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`new HashSet<>()` TAKES `nothing, an int capacity, or a Collection to copy` RETURNS `HashSet<E>` `O(1), copy O(n)`	Empty set; `new HashSet<>(list)` removes the duplicates of a list in one step.	`Set<Integer> unique = new HashSet<>(Arrays.asList(4, 4, 2));` `int n = unique.size(); // → 2`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`add(E e)` RETURNS `boolean` O(1)	Adds `e`; returns **false** if it was already present. Use the return value to detect duplicates without a separate `contains`.	`boolean added = set.add(4); // → true` `boolean dup = set.add(2); // → false`
`contains(Object o)` RETURNS `boolean` O(1)	Membership test.	`boolean has = set.contains(3); // → true`
`remove(Object o)` RETURNS `boolean` O(1)	Deletes `o`; true if it was present. Sets have no indexes, so `remove(2)` on a `Set<Integer>` removes the value 2.	`boolean r = set.remove(2); // → true` `boolean again = set.remove(2); // → false`
`size()` RETURNS `int` O(1)	Number of elements; also `isEmpty()` and `clear()`.	`int n = set.size(); // → 3`
`addAll(Collection<? extends E> c)` RETURNS `boolean` O(m)	Union, in place: the set becomes set U c.	`set.addAll(Arrays.asList(3, 4, 5)); // set → [1, 2, 3, 4, 5]`
`retainAll(Collection<?> c)` RETURNS `boolean` O(n)	Intersection, in place: keeps only elements also in `c`. Pass a `Set` so each lookup is O(1).	`set.retainAll(new HashSet<>(Arrays.asList(2, 3, 9))); // set → [2, 3]`
`removeAll(Collection<?> c)` RETURNS `boolean` O(n) or O(m)	Difference, in place: removes everything that is in `c`.	`set.removeAll(Arrays.asList(1)); // set → [2, 3]`
`containsAll(Collection<?> c)` RETURNS `boolean` O(m)	Subset test: true if every element of `c` is in the set.	`boolean sub = set.containsAll(Arrays.asList(1, 3)); // → true`

`for (E x : set)` TAKES — RETURNS not a call O(n)	Visits every element once, in no guaranteed order.	`int sum = 0;`
		`for (int x : set) sum += x;`
		`// sum → 6`

`Set.of(E... elements)` JAVA 9+ RETURNS `Set<E>` O(n)	Immutable set; repeated arguments throw `IllegalArgumentException`.	`Set<Character> vowels = Set.of('a', 'e', 'i', 'o', 'u');` `boolean v = vowels.contains('e'); // → true`

Visited set over grid cells — TEMPLATE

```
int rows = 3, cols = 4;
Set<Integer> visited = new HashSet<>();
visited.add(1 * cols + 2);                      // cell (1, 2) encoded as r * cols + c
boolean seen = visited.contains(1 * cols + 2);  // → true
int code = 1 * cols + 2;
int r = code / cols, c = code % cols;           // decode
// r + "," + c → "1,2"
boolean[][] vis = new boolean[rows][cols];      // simpler and faster when the grid size is known
vis[1][2] = true;
```

MAPS & SETS

TreeSet<E>: a sorted set

12 entries

Unique elements kept in sorted order, O(log n) per operation. The set version of `TreeMap`: same nearest-element queries, without values.

IMPORT `java.util.TreeSet`

START STATE · EVERY EXAMPLE BEGINS HERE

```
TreeSet<Integer> ts = new TreeSet<>(Arrays.asList(10, 20, 30));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`new TreeSet<>()` TAKES `nothing, a Comparator<? super E>, or a Collection` RETURNS `TreeSet<E>` O(1), copy O(n log n)	Natural order, or pass a comparator. Two elements that compare as 0 count as the **same** element, so a comparator on only part of an object silently drops ties.	`TreeSet<int[]> byFirst = new TreeSet<>((x, y) -> Integer.compare(x[0], y[0]));`
		`byFirst.add(new int[] {1, 5});`
		`boolean added = byFirst.add(new int[] {1, 9}); // → false`

`add(E e)` RETURNS `boolean` O(log n)	Adds `e`; false if an equal element exists. `remove` and `contains` are O(log n) too.	`ts.add(25); // ts → [10, 20, 25, 30]`
`first()` RETURNS `E` O(log n)	Smallest element; `last()` for the largest. Both throw `NoSuchElementException` when empty.	`int lo = ts.first(); // → 10` `int hi = ts.last(); // → 30`
`floor(E e)` RETURNS `E` O(log n)	Greatest element ≤ e, or `null`.	`Integer f = ts.floor(25); // → 20` `Integer none = ts.floor(5); // → null`
`ceiling(E e)` RETURNS `E` O(log n)	Least element ≥ e, or `null`.	`Integer c = ts.ceiling(25); // → 30`
`lower(E e)` RETURNS `E` O(log n)	Greatest element **strictly** < e, or `null`.	`Integer l = ts.lower(20); // → 10`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`higher(E e)` RETURNS `E` O(log n)	Least element **strictly >** e, or `null`.	`Integer h = ts.higher(20); // → 30` `Integer none = ts.higher(30); // → null`
`pollFirst()` RETURNS `E` O(log n)	Removes and returns the smallest, or `null` if empty; `pollLast()` for the largest.	`Integer lo = ts.pollFirst(); // → 10` `// ts → [20, 30]`
`headSet(E toElement)` RETURNS `SortedSet<E>` O(log n) to create	View of elements **<** toElement; `headSet(e, true)` includes it.	`SortedSet<Integer> h = ts.headSet(20); // → [10]`
`tailSet(E fromElement)` RETURNS `SortedSet<E>` O(log n) to create	View of elements **≥** fromElement; `tailSet(e, false)` excludes it.	`SortedSet<Integer> t = ts.tailSet(20); // → [20, 30]`
`subSet(E fromElement, E toElement)` RETURNS `SortedSet<E>` O(log n) to create	View of [from, to); `subSet(from, true, to, true)` includes both ends. Careful: `size()` of a view counts by walking it, O(size).	`SortedSet<Integer> s = ts.subSet(10, 30); // → [10, 20]`
`descendingSet()` RETURNS `NavigableSet<E>` O(1)	Reverse-order view; `descendingIterator()` for just an iterator.	`NavigableSet<Integer> d = ts.descendingSet(); // → [30, 20, 10]`

TIP **Need duplicates?**

A `TreeSet` cannot hold the same value twice. Use the `TreeMap` multiset from the TreeMap section, or store `int[] {value, index}` with a comparator that breaks ties on the index.

PART 5

Stacks, queues and heaps

Stack · java.util.Stack (legacy) · Queue · Deque · PriorityQueue<E>

STACKS, QUEUES & HEAPS

Stack: ArrayDeque used as a stack

♥ 6 entries

Last in, first out. Declare `Deque<Integer> stack = new ArrayDeque<>()` and use `push`, `pop`, `peek`. All three work on the **front** of the deque, so the top of the stack is the first element.

IMPORT `java.util.ArrayDeque, java.util.Deque`

START STATE · EVERY EXAMPLE BEGINS HERE

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.push(2);
stack.push(3);       // top is 3
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`new ArrayDeque<>()` TAKES `nothing, an int capacity, or a Collection` RETURNS `ArrayDeque<E>` O(1)	Resizable circular array; use it instead of `Stack` and `LinkedList` for stacks and queues. It does not accept `null` elements.	`Deque<Character> st = new ArrayDeque<>();` `st.push(null); // throws NullPointerException`
`push(E e)` RETURNS `void` O(1) amortised	Puts `e` on top (same as `addFirst`).	`stack.push(4); // stack → [4, 3, 2, 1]`

`pop()` RETURNS `E` O(1)	Removes and returns the top. Throws `NoSuchElementException` if empty, so check `isEmpty()` first.	`int top = stack.pop(); // → 3`
		`// stack → [2, 1]`
		`new ArrayDeque<Integer>().pop(); // throws NoSuchElementException`

`peek()` RETURNS `E` O(1)	Top element without removing it, or `null` if empty.	`int top = stack.peek(); // → 3` `Integer none = new ArrayDeque<Integer>().peek(); // → null`
`isEmpty()` RETURNS `boolean` O(1)	True if the stack has no elements; `size()` gives the count.	`boolean e = stack.isEmpty(); // → false`

`for (E x : stack)` TAKES — RETURNS not a call O(n)	Visits elements from top to bottom; `toString()` also lists the top first.	`StringBuilder sb = new StringBuilder();`
		`for (int x : stack) sb.append(x);`
		`// sb → "321"`

Monotonic stack: next greater element

TEMPLATE

```
int[] a = {2, 1, 2, 4, 3};
int[] next = new int[a.length];
Deque<Integer> st = new ArrayDeque<>();                // indexes; their values decrease from bottom to top
for (int i = a.length - 1; i >= 0; i--) {
    while (!st.isEmpty() && a[st.peek()] <= a[i]) st.pop();
    next[i] = st.isEmpty() ? -1 : a[st.peek()];
    st.push(i);
}
// next → [4, 2, 4, -1, -1]
```

Each index is pushed and popped at most once, so the whole loop is O(n). The same shape solves daily temperatures, stock span and largest rectangle in a histogram.

Balanced brackets

TEMPLATE

```
String in = "{[()]}()";
Deque<Character> st = new ArrayDeque<>();
boolean ok = true;
for (char ch : in.toCharArray()) {
    if (ch == '(' || ch == '[' || ch == '{') st.push(ch);
    else {
        if (st.isEmpty()) { ok = false; break; }
        char open = st.pop();
        if ((ch == ')' && open != '(') || (ch == ']' && open != '[') || (ch == '}' && open != '{')) { ok = false; break; }
    }
}
ok = ok && st.isEmpty();
// ok → true
```

STACKS, QUEUES & HEAPS

java.util.Stack (legacy)

6 entries

Older class that many tutorials use. It works, but it is synchronised (slower), its iteration order is the opposite of `ArrayDeque`, and its exceptions differ. Know it so you can read other people's code.

IMPORT `java.util.Stack`

START STATE · EVERY EXAMPLE BEGINS HERE

```
Stack<Integer> st = new Stack<>();
st.push(1);
st.push(2);
st.push(3);
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`push(E item)` RETURNS `E` `O(1) amortised`	Pushes and returns `item` (`ArrayDeque.push` returns `void`).	`int r = st.push(4);  // → 4`
`pop()` RETURNS `E` `O(1)`	Removes the top. Throws `EmptyStackException`, not `NoSuchElementException`, when empty.	`int top = st.pop();  // → 3` `new Stack<Integer>().pop();  // throws EmptyStackException`
`peek()` RETURNS `E` `O(1)`	Top without removing. Throws `EmptyStackException` when empty, where `ArrayDeque.peek` returns `null`.	`int top = st.peek();  // → 3` `new Stack<Integer>().peek();  // throws EmptyStackException`
`empty()` RETURNS `boolean` `O(1)`	Same as `isEmpty()`.	`boolean e = st.empty();  // → false`
`search(Object o)` RETURNS `int` `O(n)`	1-based distance from the top, or −1 if absent.	`int d = st.search(1);  // → 3`
`toString()` RETURNS `String` `O(n)`	Lists **bottom to top**, the opposite of `ArrayDeque`. `Stack` extends `Vector`, so `get(i)` also works (index 0 is the bottom).	`String t = st.toString();  // → "[1, 2, 3]"` `int bottom = st.get(0);  // → 1`

STACKS, QUEUES & HEADS

Queue: ArrayDeque used as a queue

7 entries

First in, first out: the order for BFS. Declare `Queue<Integer> q = new ArrayDeque<>()`. Each operation comes in two forms, one that throws on failure and one that returns a special value.

IMPORT `java.util.ArrayDeque, java.util.Queue`

START STATE · EVERY EXAMPLE BEGINS HERE

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(1);
q.offer(2);
q.offer(3);          // head is 1
```

OPERATION	THROWS AN EXCEPTION IF IT CANNOT	RETURNS A SPECIAL VALUE INSTEAD
Add at the tail	`add(e)`	`offer(e)` → `false`
Remove the head	`remove()` → `NoSuchElementException`	`poll()` → `null`
Look at the head	`element()` → `NoSuchElementException`	`peek()` → `null`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`offer(E e)` RETURNS `boolean` `O(1) amortised`	Adds at the tail. Returns true (false only for a full bounded queue).	`q.offer(4);  // q → [1, 2, 3, 4]`
`add(E e)` RETURNS `boolean` `O(1) amortised`	Same as `offer`, but a full bounded queue throws instead of returning false. For `ArrayDeque` the two are identical.	`boolean ok = q.add(4);  // → true`

`poll()` RETURNS `E` `O(1)`	Removes and returns the head, or `null` if empty.	`int head = q.poll();  // → 1`
		`// q → [2, 3]`
		`Integer none = new ArrayDeque<Integer>().poll();  // → null`

`remove()` RETURNS `E` `O(1)`	Removes and returns the head; throws `NoSuchElementException` if empty. With an argument, `remove(Object o)` deletes that element in O(n).	`int head = q.remove();  // → 1`
`peek()` RETURNS `E` `O(1)`	Head without removing, or `null` if empty.	`int head = q.peek();  // → 1`
`element()` RETURNS `E` `O(1)`	Head without removing; throws `NoSuchElementException` if empty.	`int head = q.element();  // → 1` `new ArrayDeque<Integer>().element();  // throws NoSuchElementException`
`size()` RETURNS `int` `O(1)`	Number of elements. In level-by-level BFS, read it **once** before the inner loop, because the loop adds the next level to the same queue.	`int n = q.size();  // → 3`

BFS on a grid: shortest path in steps

TEMPLATE

```
int[][] grid = {
    {0, 0, 1},
    {1, 0, 1},
    {1, 0, 0}};                                    // 1 = wall
int n = grid.length, m = grid[0].length;
int[][] dist = new int[n][m];
for (int[] row : dist) Arrays.fill(row, -1);
int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
Queue<int[]> q = new ArrayDeque<>();
q.offer(new int[] {0, 0});
dist[0][0] = 0;
while (!q.isEmpty()) {
    int[] cur = q.poll();
    for (int[] d : dirs) {
        int r = cur[0] + d[0], c = cur[1] + d[1];
        if (r < 0 || r >= n || c < 0 || c >= m || grid[r][c] == 1 || dist[r][c] != -1) continue;
        dist[r][c] = dist[cur[0]][cur[1]] + 1;          // mark when you enqueue, not when you dequeue
        q.offer(new int[] {r, c});
    }
}
int steps = dist[2][2];  // → 4
```

BFS level by level

TEMPLATE

```
List<List<Integer>> levels = new ArrayList<>();
Map<Integer, List<Integer>> children = new HashMap<>();
children.put(1, Arrays.asList(2, 3));
children.put(2, Arrays.asList(4));
```

```java
Queue<Integer> q = new ArrayDeque<>();
q.offer(1);
while (!q.isEmpty()) {
    List<Integer> level = new ArrayList<>();
    for (int size = q.size(); size > 0; size--) {       // size read once per level
        int u = q.poll();
        level.add(u);
        for (int v : children.getOrDefault(u, Collections.emptyList())) q.offer(v);
    }
    levels.add(level);
}
// levels → [[1], [2, 3], [4]]
```

STACKS, QUEUES & HEAPS

Deque: both ends

8 entries

`ArrayDeque` adds, removes and reads at both ends in O(1). The stack and queue methods above are just names for some of these.

IMPORT `java.util.ArrayDeque, java.util.Deque`

START STATE · EVERY EXAMPLE BEGINS HERE

```java
Deque<Integer> dq = new ArrayDeque<>(Arrays.asList(1, 2, 3));   // first 1, last 3
```

	FRONT, THROWS	FRONT, RETURNS NULL/FALSE	BACK, THROWS	BACK, RETURNS NULL/FALSE
Add	`addFirst(e)` = `push(e)`	`offerFirst(e)`	`addLast(e)` = `add(e)`	`offerLast(e)` = `offer(e)`
Remove	`removeFirst()` = `pop()` = `remove()`	`pollFirst()` = `poll()`	`removeLast()`	`pollLast()`
Look	`getFirst()` = `element()`	`peekFirst()` = `peek()`	`getLast()`	`peekLast()`

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`offerFirst(E e)` RETURNS `boolean` O(1) amortised	Adds at the front. `addFirst` does the same and returns `void`.	`dq.offerFirst(0);  // dq → [0, 1, 2, 3]`
`offerLast(E e)` RETURNS `boolean` O(1) amortised	Adds at the back (same as `offer`). `addLast` returns `void`.	`dq.offerLast(4);  // dq → [1, 2, 3, 4]`
`pollFirst()` RETURNS `E` O(1)	Removes and returns the front, or `null` if empty. `removeFirst` throws instead.	`int f = dq.pollFirst();  // → 1`
`pollLast()` RETURNS `E` O(1)	Removes and returns the back, or `null` if empty. `removeLast` throws instead.	`int l = dq.pollLast();  // → 3` `// dq → [1, 2]`
`peekFirst()` RETURNS `E` O(1)	Front without removing, or `null`. `getFirst` throws instead.	`int f = dq.peekFirst();  // → 1`
`peekLast()` RETURNS `E` O(1)	Back without removing, or `null`. `getLast` throws instead.	`int l = dq.peekLast();  // → 3`
`descendingIterator()` RETURNS `Iterator<E>` O(1)	Iterates from back to front.	`Iterator<Integer> it = dq.descendingIterator();` `int first = it.next();  // → 3`
`removeFirstOccurrence(Object o)` RETURNS `boolean` O(n)	Deletes the first element equal to `o`; `removeLastOccurrence` for the last.	`dq.removeFirstOccurrence(2);  // dq → [1, 3]`

Monotonic deque: sliding window maximum (LeetCode 239)

TEMPLATE

```java
int[] a = {1, 3, -1, -3, 5, 3, 6, 7};
int k = 3;
int[] res = new int[a.length - k + 1];
Deque<Integer> dq = new ArrayDeque<>();          // indexes; values decrease from front to back
for (int i = 0; i < a.length; i++) {
    if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();        // front left the window
    while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();    // can never be the max again
    dq.offerLast(i);
    if (i >= k - 1) res[i - k + 1] = a[dq.peekFirst()];
}
// res → [3, 3, 5, 5, 6, 7]
```

STACKS, QUEUES & HEAPS

PriorityQueue<E>: a binary heap

9 entries

`poll()` always returns the smallest element according to the ordering: a **min-heap** by default. Pass a comparator for a max-heap or to order arrays and objects. Insert and remove are O(log n); looking at the top is O(1).

IMPORT `java.util.PriorityQueue`

START STATE · EVERY EXAMPLE BEGINS HERE

```
PriorityQueue<Integer> pq = new PriorityQueue<>(Arrays.asList(5, 1, 8, 3));   // min-heap
```

CREATING

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`new PriorityQueue<>()`
TAKES `nothing, an int capacity, or a Collection`
RETURNS `PriorityQueue<E>`
`O(1), from a collection O(n)`
	Min-heap in natural order. Built from a collection, it heapifies in O(n).	`PriorityQueue<Integer> minHeap = new PriorityQueue<>();`
		`minHeap.add(4);`
		`minHeap.add(2);`
		`int smallest = minHeap.peek();  // → 2`

`new PriorityQueue<>(Comparator<? super E> comparator)`
RETURNS `PriorityQueue<E>` `O(1)`
	Heap ordered by the comparator: `Collections.reverseOrder()` gives a max-heap; a lambda orders arrays such as `int[] {distance, node}`. `new PriorityQueue<>(capacity, comparator)` also exists.	`PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());`
		`maxHeap.addAll(Arrays.asList(5, 1, 8));`
		`int largest = maxHeap.peek();  // → 8`
		`PriorityQueue<int[]> byDist = new PriorityQueue<>((x, y) -> Integer.compare(x[0], y[0]));`
		`byDist.add(new int[] {7, 0});`
		`byDist.add(new int[] {2, 1});`
		`int[] nearest = byDist.poll();  // → [2, 1]`

USING THE HEAP

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`add(E e)` RETURNS `boolean` `O(log n)`	Inserts `e`. `offer` is identical. Returns true.	`pq.add(0);` `int min = pq.peek();  // → 0`
`peek()` RETURNS `E` `O(1)`	Smallest element without removing it, or `null` if empty. `element()` throws instead.	`int min = pq.peek();  // → 1`
`poll()` RETURNS `E` `O(log n)`	Removes and returns the smallest element, or `null` if empty. `remove()` with no argument throws instead.	`int a = pq.poll();  // → 1` `int b = pq.poll();  // → 3`
`size()` RETURNS `int` `O(1)`	Number of elements; also `isEmpty()` and `clear()`.	`int n = pq.size();  // → 4`
`remove(Object o)` RETURNS `boolean` `O(n)`	Deletes one element equal to `o`. It searches linearly, so O(n). For frequent deletions, use a `TreeMap` multiset or lazy deletion.	`boolean r = pq.remove(8);  // → true` `int n = pq.size();  // → 3`
`contains(Object o)` RETURNS `boolean` `O(n)`	Linear search.	`boolean has = pq.contains(3);  // → true`

`toString()`
RETURNS `String` `O(n)`
	Shows the internal heap array, **not** sorted order; iteration is the same. To read in order, `poll()` until empty.	`String raw = pq.toString();  // → "[1, 3, 8, 5]"`
		`StringBuilder sorted = new StringBuilder();`
		`while (!pq.isEmpty())`
		`sorted.append(pq.poll()).append(' ');`
		`// sorted → "1 3 5 8 "`

Top k: keep a min-heap of size k

TEMPLATE

```
int[] nums = {3, 2, 1, 5, 6, 4};
int k = 2;
PriorityQueue<Integer> heap = new PriorityQueue<>();        // holds the k largest seen so far
for (int x : nums) {
    heap.offer(x);
    if (heap.size() > k) heap.poll();                       // drop the smallest
}
int kthLargest = heap.peek();  // → 5
```

O(n log k) time and O(k) memory. For the k smallest, use a max-heap of size k.

Merge k sorted lists with a heap of {value, list, index} TEMPLATE

```
int[][] lists = {{1, 4, 7}, {2, 5}, {3, 6, 9}};
PriorityQueue<int[]> heap = new PriorityQueue<>((x, y) -> Integer.compare(x[0], y[0]));
for (int i = 0; i < lists.length; i++) heap.offer(new int[] {lists[i][0], i, 0});
List<Integer> merged = new ArrayList<>();
while (!heap.isEmpty()) {
    int[] t = heap.poll();
    merged.add(t[0]);
    int li = t[1], idx = t[2] + 1;
    if (idx < lists[li].length) heap.offer(new int[] {lists[li][idx], li, idx});
}
// merged → [1, 2, 3, 4, 5, 6, 7, 9]
```

PART 6

Ordering, streams and conversions

Comparator<T> and Comparable<T> · Streams · BitSet · Converting between types

ORDERING & STREAMS Comparator<T> and Comparable<T>

♥ 7 entries

A comparator answers "does a come before b?" with an `int`: negative means a first, 0 means equal, positive means b first. Sorting methods, `TreeMap`, `TreeSet` and `PriorityQueue` all take one.

IMPORT `java.util.Comparator`

START STATE · EVERY EXAMPLE BEGINS HERE

```
List<String> words = new ArrayList<>(Arrays.asList("pear", "fig", "apple", "kiwi"));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`compare(T a, T b)` RETURNS `int` O(1)	The single method of `Comparator<T>`, usually written as a lambda `(a, b) -> ...`.	`Comparator<String> byLen = (a, b) -> Integer.compare(a.length(), b.length());`
		`int c = byLen.compare("fig", "pear");  // → -1`
		`words.sort(byLen);  // words → [fig, pear, kiwi, apple]`

`compareTo(T other)` RETURNS `int` O(1)	The single method of `Comparable<T>`: the natural order of `Integer`, `Long`, `String`, `Character`. Used whenever you pass no comparator.	`int c = "fig".compareTo("kiwi");  // → -5` `words.sort(null);  // words → [apple, fig, kiwi, pear]`

`Comparator.comparingInt(ToIntFunction<? super T> keyExtractor)` RETURNS `Comparator<T>` O(1)	Compares by an `int` key. Also `comparingLong`, `comparingDouble`, and `comparing(Function)` for any comparable key.	`words.sort(Comparator.comparingInt(String::length));  // words → [fig, pear, kiwi, apple]`
		`int[][] pts = {{3, 1}, {1, 2}};`
		`Arrays.sort(pts, Comparator.comparingInt(p -> p[0]));  // pts → [[1, 2], [3, 1]]`

`thenComparing(Comparator<? super T> other)` RETURNS `Comparator<T>` O(1)	Tie-breaker used when the first comparison returns 0. Also `thenComparingInt(keyExtractor)` and `thenComparing(Function)`.	`words.sort(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder()));` `// words → [fig, kiwi, pear, apple]`

`reversed()` RETURNS `Comparator<T>` O(1)	The same comparator, backwards. After a lambda, Java may fail to infer the type; give the parameter a type: `Comparator.comparingInt((int[] p) -> p[0]).reversed()`.	`words.sort(Comparator.comparingInt(String::length).reversed());  // words → [apple, pear, kiwi, fig]`
		`int[][] pts = {{1, 2}, {3, 1}};`
		`Arrays.sort(pts, Comparator.comparingInt((int[] p) -> p[0]).reversed());  // pts → [[3, 1], [1, 2]]`

`Comparator.reverseOrder()` RETURNS `Comparator<T>` O(1)	Descending natural order; `Comparator.naturalOrder()` is ascending. Same as `Collections.reverseOrder()`.	`words.sort(Comparator.reverseOrder());  // words → [pear, kiwi, fig, apple]`

`Integer.compare(int x, int y)` RETURNS `int` O(1)	Write comparator bodies with this. `(a, b) -> a - b` overflows when the values are far apart and then sorts wrongly.	`Comparator<Integer> bad = (a, b) -> a - b;`
		`int wrong = bad.compare(Integer.MIN_VALUE, 1);  // → 2147483647`
		`int right = Integer.compare(Integer.MIN_VALUE, 1);  // → -1`

Sort intervals by start, then by end descending TEMPLATE

```
int[][] iv = {{1, 4}, {0, 2}, {1, 3}};
Arrays.sort(iv, (x, y) -> x[0] != y[0] ? Integer.compare(x[0], y[0]) : Integer.compare(y[1], x[1]));
// iv → [[0, 2], [1, 4], [1, 3]]
```

Your own class with a natural order

TEMPLATE

```
class Task implements Comparable<Task> {
    final String name;
    final int priority;
    Task(String name, int priority) { this.name = name; this.priority = priority; }
    @Override public int compareTo(Task o) { return Integer.compare(priority, o.priority); }
    @Override public String toString() { return name; }
}

PriorityQueue<Task> tasks = new PriorityQueue<>();
tasks.add(new Task("write", 2));
tasks.add(new Task("read", 1));
Task first = tasks.poll();  // → read
```

In a solution file, put the class at the top level or as a `static class` inside `Solution`.

ORDERING & STREAMS

Streams

11 entries

Streams give one-line totals, filters and groupings. They are slower than a plain loop, so keep them out of the hot path of an O(n log n) solution with n = 10^5 or more; use them for setup and conversions.

IMPORT `java.util.stream.*`

START STATE · EVERY EXAMPLE BEGINS HERE

```
List<Integer> nums = new ArrayList<>(Arrays.asList(4, 1, 3, 1, 5));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`IntStream.range(int startInclusive, int endExclusive)` RETURNS `IntStream` O(n)	The numbers start..end−1; `rangeClosed` includes the end.	`int s = IntStream.range(0, 5).sum();  // → 10` `int t = IntStream.rangeClosed(1, 5).sum();  // → 15`
`mapToInt(ToIntFunction<? super T> mapper)` RETURNS `IntStream` O(n)	Turns a `Stream<Integer>` into an `IntStream`, which has `sum()`, `max()`, `min()`, `average()` and `toArray()`.	`int total = nums.stream().mapToInt(Integer::intValue).sum();  // → 14`
`boxed()` RETURNS `Stream<Integer>` O(n)	Turns an `IntStream` into a `Stream<Integer>`, needed before `collect`.	`List<Integer> list = IntStream.of(3, 1, 2).boxed().collect(Collectors.toList());  // → [3, 1, 2]`
`max()` RETURNS `OptionalInt` O(n)	On an `IntStream`, returns an `OptionalInt`: call `getAsInt()` (throws `NoSuchElementException` if the stream was empty) or `orElse(default)`. On a `Stream<T>`, `max(comparator)` returns an `Optional<T>`; call `get()`.	`int mx = nums.stream().mapToInt(Integer::intValue).max().getAsInt();  // → 5` `int none = IntStream.empty().max().orElse(-1);  // → -1`
`filter(Predicate<? super T> predicate)` RETURNS `Stream<T>` O(n)	Keeps the elements that match.	`List<Integer> odd = nums.stream().filter(x -> x % 2 == 1).collect(Collectors.toList());  // → [1, 3, 1, 5]`
`map(Function<? super T, ? extends R> mapper)` RETURNS `Stream<R>` O(n)	Transforms each element.	`List<String> s = nums.stream().map(x -> "n" + x).collect(Collectors.toList());  // → [n4, n1, n3, n1, n5]`
`sorted()` RETURNS `Stream<T>` O(n log n)	Sorted stream; `sorted(comparator)` for other orders. Also `distinct()` and `limit(n)`.	`List<Integer> top2 = nums.stream().sorted(Comparator.reverseOrder()).limit(2).collect(Collectors.toList());  // → [5, 4]` `long unique = nums.stream().distinct().count();  // → 4`
`anyMatch(Predicate<? super T> predicate)` RETURNS `boolean` O(n)	True if some element matches; stops at the first. Also `allMatch` and `noneMatch`.	`boolean big = nums.stream().anyMatch(x -> x > 4);  // → true`
`collect(Collector<? super T, A, R> collector)` RETURNS `R` O(n)	Gathers the results. Common collectors: `Collectors.toList()`, `toSet()`, `joining(sep)`, `groupingBy(f)`, `groupingBy(f, counting())`, `toMap(keyFn, valueFn)`.	`String joined = nums.stream().map(String::valueOf).collect(Collectors.joining(","));  // → "4,1,3,1,5"` `Map<Integer, Long> freq = nums.stream().collect(Collectors.groupingBy(x -> x, Collectors.counting()));  // → {1=2, 3=1, 4=1, 5=1}`
`toArray()` RETURNS `int[]` O(n)	On an `IntStream`, returns an `int[]`: the standard `List<Integer>` → `int[]` conversion.	`int[] arr = nums.stream().mapToInt(Integer::intValue).toArray();  // → [4, 1, 3, 1, 5]`
`toList()` JAVA 16+ RETURNS `List<T>` O(n)	Unmodifiable list straight from a stream. In Java 8 write `collect(Collectors.toList())`.	`List<Integer> l = nums.stream().filter(x -> x > 2).toList();  // → [4, 3, 5]`

ORDERING & STREAMS

BitSet

8 entries

A growable array of bits: 1 bit per flag where `boolean[]` uses a byte. Useful for sieves and visited flags over large ranges, and for fast set operations on bit masks longer than 64.

IMPORT `java.util.BitSet`

START STATE · EVERY EXAMPLE BEGINS HERE

```
BitSet bs = new BitSet(16);
bs.set(1);
bs.set(4);
bs.set(5);
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`new BitSet(int nbits)` RETURNS `BitSet` `O(nbits / 64)`	All bits start false. It grows automatically if you set a higher index.	`BitSet b = new BitSet(100);` `int on = b.cardinality(); // → 0`
`set(int bitIndex)` RETURNS `void` `O(1)`	Sets the bit to true. `set(from, to)` sets a range; `set(i, false)` clears.	`bs.set(7); // bs → {1, 4, 5, 7}`
`get(int bitIndex)` RETURNS `boolean` `O(1)`	Value of the bit.	`boolean b = bs.get(4); // → true`
`clear(int bitIndex)` RETURNS `void` `O(1)`	Sets the bit to false; `clear()` clears all of them.	`bs.clear(4); // bs → {1, 5}`
`flip(int bitIndex)` RETURNS `void` `O(1)`	Toggles the bit.	`bs.flip(0); // bs → {0, 1, 4, 5}`
`cardinality()` RETURNS `int` `O(n / 64)`	Number of true bits.	`int n = bs.cardinality(); // → 3`
`nextSetBit(int fromIndex)` RETURNS `int` `O(n / 64)`	First true index ≥ fromIndex, or −1. Loop over all set bits with `for (int i = bs.nextSetBit(0); i >= 0; i = bs.nextSetBit(i + 1))`. Also `nextClearBit`.	`int i = bs.nextSetBit(2); // → 4` `int none = bs.nextSetBit(6); // → -1`

`and(BitSet set)` RETURNS `void` `O(n / 64)`	In-place AND with another bit set; also `or`, `xor` and `andNot`.	`BitSet other = new BitSet();`
		`other.set(4);`
		`other.set(9);`
		`bs.and(other); // bs → {4}`

ORDERING & STREAMS

Converting between types

20 entries

The conversions you type most often. Each row is an expression; the result column is what Java prints for it.

START STATE · EVERY EXAMPLE BEGINS HERE

```
int[] arr = {3, 1, 2};
List<Integer> list = new ArrayList<>(Arrays.asList(3, 1, 2));
String s = "cab";
```

EXPRESSION	RESULT	WHY IT MATTERS
`Arrays.stream(arr).boxed().collect(Collectors.toList())`	`[3, 1, 2]`	`int[]` → `List<Integer>`. A loop with `list.add(x)` does the same and is faster.
`list.stream().mapToInt(Integer::intValue).toArray()`	`[3, 1, 2]`	`List<Integer>` → `int[]`.
`list.toArray(new Integer[0])`	`[3, 1, 2]`	`List<Integer>` → `Integer[]` (boxed, so it can be sorted with a comparator).
`Arrays.stream(arr).boxed().toArray(Integer[]::new)`	`[3, 1, 2]`	`int[]` → `Integer[]`.
`new HashSet<>(list)`	`[1, 2, 3]`	`List` → `Set` (drops duplicates).
`new ArrayList<>(new TreeSet<>(list))`	`[1, 2, 3]`	Sorted list of the distinct values.
`s.toCharArray()`	`[c, a, b]`	`String` → `char[]`.
`new String(new char[] {'h', 'i'})`	`"hi"`	`char[]` → `String`; `String.valueOf(chars)` is the same.
`String.valueOf(42)`	`"42"`	`int` → `String`; `Integer.toString(42)` and `"" + 42` also work.
`Integer.parseInt("42")`	`42`	`String` → `int`; `Long.parseLong` for `long`.
`String.valueOf('x')`	`"x"`	`char` → `String`.
`"x".charAt(0)`	`'x'`	One-character `String` → `char`.

EXPRESSION	RESULT	WHY IT MATTERS
`(int) 'A'`	`65`	`char` → its code.
`(char) 66`	`'B'`	Code → `char`.
`Character.getNumericValue('7')`	`7`	Digit `char` → `int` (or `'7' - '0'`).
`(long) Integer.MAX_VALUE + 1`	`2147483648`	Widening `int` → `long` is automatic in assignments; cast before arithmetic.
`(int) 3_000_000_000L`	`-1294967296`	Narrowing `long` → `int` keeps the low 32 bits silently; `Math.toIntExact` throws instead.
`String.join(",", Arrays.asList("a", "b"))`	`"a,b"`	`List<String>` → one `String`.
`list.stream().map(String::valueOf).collect(Collectors.joining(" "))`	`"3 1 2"`	`List<Integer>` → space-separated text, e.g. for output.
`Arrays.asList(s.split(""))`	`[c, a, b]`	`String` → `List<String>` of single characters.

PART 7

Structures LeetCode gives you or you write yourself

ListNode · TreeNode · Graphs · Union-Find (disjoint set union) · Trie (prefix tree) · Fenwick tree (binary indexed tree)

BUILD IT YOURSELF — ListNode: singly linked list

♥ 3 entries

Linked-list problems hand you a `ListNode` with two public **fields**, not methods. Everything is done by following and rewiring `next` pointers.

YOU WRITE THIS · THE EXAMPLES USE IT

```
class ListNode {                        // exactly as LeetCode defines it
    int val;
    ListNode next;
    ListNode() {}
    ListNode(int val) { this.val = val; }
    ListNode(int val, ListNode next) { this.val = val; this.next = next; }
}

static String str(ListNode h) {         // helper for the examples: "1 -> 2 -> 3"
    StringBuilder sb = new StringBuilder();
    for (ListNode p = h; p != null; p = p.next) sb.append(p.val).append(p.next != null ? " -> " : "");
    return sb.toString();
}
```

START STATE · EVERY EXAMPLE BEGINS HERE

```
ListNode head = new ListNode(1, new ListNode(2, new ListNode(3)));   // 1 -> 2 -> 3
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`node.val` TAKES field: no parentheses RETURNS `int` O(1)	The value stored in the node.	`int v = head.val;  // → 1`
`node.next` TAKES field: no parentheses RETURNS `ListNode` O(1)	The following node, or `null` after the tail. Check `node != null` before reading `node.val` or `node.next`.	`int second = head.next.val;  // → 2` `ListNode end = head.next.next.next;  // → null`
`new ListNode(int val, ListNode next)` RETURNS `ListNode` O(1)	A new node in front of `next`; `new ListNode(v)` has `next = null`.	`ListNode front = new ListNode(0, head);` `String t = str(front);  // → "0 -> 1 -> 2 -> 3"`

Linked-list moves you will reuse — TEMPLATE

```
int len = 0;                                          // length
for (ListNode p = head; p != null; p = p.next) len++;
// len → 3

ListNode dummy = new ListNode(0), tail = dummy;       // build with a dummy head
for (int v : new int[] {4, 5, 6}) { tail.next = new ListNode(v); tail = tail.next; }
// str(dummy.next) → "4 -> 5 -> 6"

ListNode slow = head, fast = head;                    // middle: fast moves two steps
while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
// slow.val → 2

boolean cycle = false;                                // Floyd cycle check
for (ListNode a = head, b = head; b != null && b.next != null; ) {
```

```
    a = a.next; b = b.next.next;
    if (a == b) { cycle = true; break; }
}
// cycle → false

ListNode prev = null, cur = head;                        // reverse in place
while (cur != null) { ListNode nxt = cur.next; cur.next = prev; prev = cur; cur = nxt; }
// str(prev) → "3 -> 2 -> 1"
```

BUILD IT YOURSELF

TreeNode: binary tree

4 entries

Tree problems hand you a `TreeNode` with fields `val`, `left`, `right`. A missing child is `null`, so most recursive solutions start with `if (node == null) return ...;`.

YOU WRITE THIS · THE EXAMPLES USE IT

```
class TreeNode {                          // exactly as LeetCode defines it
    int val;
    TreeNode left, right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) { this.val = val; this.left = left; this.right = right; }
}

static int height(TreeNode n) { return n == null ? 0 : 1 + Math.max(height(n.left), height(n.right)); }

static void inorder(TreeNode n, List<Integer> out) {      // left, node, right: sorted order for a BST
    if (n == null) return;
    inorder(n.left, out);
    out.add(n.val);
    inorder(n.right, out);
}
```

START STATE · EVERY EXAMPLE BEGINS HERE

```
//        4
//       / \
//      2   6
//     / \
//    1   3
TreeNode root = new TreeNode(4, new TreeNode(2, new TreeNode(1), new TreeNode(3)), new TreeNode(6));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`node.val` TAKES field: no parentheses RETURNS `int` O(1)	Value in the node.	`int v = root.val;  // → 4`

`node.left`
TAKES field: no parentheses
RETURNS `TreeNode` O(1)
	Left child or `null`; `node.right` for the right child. A leaf has both `null`.	`int l = root.left.val;  // → 2`
		`boolean leaf = root.right.left == null &&`
		`root.right.right == null;  // → true`

`height(TreeNode n)` TAKES `TreeNode n (you write this method)` RETURNS `int` O(n)	Recursive depth: the template for most tree questions (max depth, diameter, balanced check).	`int h = height(root);  // → 3`
`inorder(TreeNode n, List<Integer> out)` TAKES `TreeNode n, List<Integer> out (you write this method)` RETURNS `void` O(n)	Left, node, right. For a binary search tree this visits values in sorted order.	`List<Integer> out = new ArrayList<>();` `inorder(root, out);  // out → [1, 2, 3, 4, 6]`

Level-order traversal with a queue

TEMPLATE

```
List<List<Integer>> levels = new ArrayList<>();
Queue<TreeNode> q = new ArrayDeque<>();
q.offer(root);
while (!q.isEmpty()) {
    List<Integer> level = new ArrayList<>();
    for (int size = q.size(); size > 0; size--) {
        TreeNode n = q.poll();
        level.add(n.val);
        if (n.left != null) q.offer(n.left);             // ArrayDeque rejects null: check first
        if (n.right != null) q.offer(n.right);
    }
    levels.add(level);
}
// levels → [[4], [2, 6], [1, 3]]
```

Iterative inorder with a stack (no recursion depth limit)

TEMPLATE

```
List<Integer> out = new ArrayList<>();
Deque<TreeNode> st = new ArrayDeque<>();
TreeNode cur = root;
while (cur != null || !st.isEmpty()) {
    while (cur != null) { st.push(cur); cur = cur.left; }
    cur = st.pop();
    out.add(cur.val);
    cur = cur.right;
}
// out → [1, 2, 3, 4, 6]
```

BUILD IT YOURSELF

Graphs: adjacency lists, DFS, Dijkstra, topological order

Inputs usually arrive as an edge list `int[][] edges` and a vertex count `n`. Convert to an adjacency list first; a list of lists is the most common form.

Building an adjacency list

TEMPLATE

```
int n = 4;
int[][] edges = {{0, 1}, {0, 2}, {2, 3}};
List<List<Integer>> adj = new ArrayList<>();
for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
for (int[] e : edges) {
    adj.get(e[0]).add(e[1]);
    adj.get(e[1]).add(e[0]);                    // omit this line for a directed graph
}
// adj → [[1, 2], [0], [0, 3], [2]]

List<Integer>[] g = new ArrayList[n];           // array form: one unchecked warning, slightly faster
for (int i = 0; i < n; i++) g[i] = new ArrayList<>();
for (int[] e : edges) { g[e[0]].add(e[1]); g[e[1]].add(e[0]); }
int degree0 = g[0].size();  // → 2
```

Iterative DFS and connected components

TEMPLATE

```
int n = 5;
int[][] edges = {{0, 1}, {1, 2}, {3, 4}};
List<List<Integer>> adj = new ArrayList<>();
for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
for (int[] e : edges) { adj.get(e[0]).add(e[1]); adj.get(e[1]).add(e[0]); }
boolean[] seen = new boolean[n];
int components = 0;
for (int s = 0; s < n; s++) {
    if (seen[s]) continue;
    components++;
    Deque<Integer> st = new ArrayDeque<>();
    st.push(s);
    seen[s] = true;
    while (!st.isEmpty()) {
        int u = st.pop();
        for (int v : adj.get(u)) if (!seen[v]) { seen[v] = true; st.push(v); }
    }
}
// components → 2
```

Recursive DFS is shorter, but Java's default stack overflows somewhere around 10^4–10^5 frames; an explicit stack has no such limit.

Dijkstra with a PriorityQueue (non-negative weights)

TEMPLATE

```
int n = 4;
int[][] edges = {{0, 1, 4}, {0, 2, 1}, {2, 1, 2}, {1, 3, 1}};   // from, to, weight (directed)
List<List<int[]>> g = new ArrayList<>();
for (int i = 0; i < n; i++) g.add(new ArrayList<>());
for (int[] e : edges) g.get(e[0]).add(new int[] {e[1], e[2]});
long[] dist = new long[n];
Arrays.fill(dist, Long.MAX_VALUE);
dist[0] = 0;
PriorityQueue<long[]> pq = new PriorityQueue<>((x, y) -> Long.compare(x[0], y[0]));   // {distance, node}
pq.offer(new long[] {0, 0});
while (!pq.isEmpty()) {
    long[] top = pq.poll();
    int u = (int) top[1];
    if (top[0] > dist[u]) continue;                 // stale entry: a shorter path was found already
    for (int[] e : g.get(u)) {
        if (dist[u] + e[1] < dist[e[0]]) {
            dist[e[0]] = dist[u] + e[1];
```

```
            pq.offer(new long[] {dist[e[0]], e[0]});
        }
    }
}
// dist → [0, 3, 1, 4]
```

O((V + E) log V). Skipping stale entries replaces the “decrease key” operation that `PriorityQueue` does not have.

Topological order (Kahn's algorithm)

TEMPLATE

```
int n = 4;
int[][] edges = {{0, 1}, {0, 2}, {1, 3}, {2, 3}};      // u must come before v
List<List<Integer>> adj = new ArrayList<>();
for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
int[] indeg = new int[n];
for (int[] e : edges) { adj.get(e[0]).add(e[1]); indeg[e[1]]++; }
Queue<Integer> q = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
List<Integer> order = new ArrayList<>();
while (!q.isEmpty()) {
    int u = q.poll();
    order.add(u);
    for (int v : adj.get(u)) if (--indeg[v] == 0) q.offer(v);
}
// order → [0, 1, 2, 3]
boolean hasCycle = order.size() < n;  // → false
```

BUILD IT YOURSELF · Union-Find (disjoint set union)

♥ 4 entries

Answers “are x and y in the same group?” while groups merge. With path compression and union by size, each call is effectively O(1) (inverse Ackermann, written α(n)). Use it for connected components, Kruskal's minimum spanning tree and cycle detection in undirected graphs.

YOU WRITE THIS · THE EXAMPLES USE IT

```
static class DSU {
    int[] parent, size;
    int groups;
    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        groups = n;
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }
    int find(int x) {                              // root of x's group, with path halving
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
    boolean union(int a, int b) {                  // false if already in the same group
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        groups--;
        return true;
    }
}
```

START STATE · EVERY EXAMPLE BEGINS HERE

```
DSU d = new DSU(5);
d.union(0, 1);
d.union(1, 2);
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`new DSU(int n)` RETURNS DSU O(n)	n elements 0..n−1, each in its own group.	`DSU fresh = new DSU(3);` `int g = fresh.groups;  // → 3`
`find(int x)` RETURNS int O(α(n))	Representative (root) of x's group. Two elements are connected exactly when their roots are equal.	`boolean together = d.find(0) == d.find(2);  // → true` `boolean apart = d.find(0) == d.find(3);  // → false`
`union(int a, int b)` RETURNS boolean O(α(n))	Merges the two groups. Returns false if they were already one group, which in an undirected graph means the edge closes a cycle.	`boolean merged = d.union(3, 4);  // → true` `boolean cycle = !d.union(0, 2);  // → true`
`groups / size[find(x)]` TAKES — (fields) RETURNS int O(α(n))	Number of groups, and the size of x's group.	`int g = d.groups;  // → 3` `int sz = d.size[d.find(0)];  // → 3`

BUILD IT YOURSELF

Trie (prefix tree)

3 entries

Stores words character by character so that “is there a word with this prefix?” costs O(length of the prefix), independent of how many words are stored (LeetCode 208).

YOU WRITE THIS · THE EXAMPLES USE IT

```
static class Trie {
    Trie[] next = new Trie[26];
    boolean end;                                  // a word ends here
    void insert(String w) {
        Trie t = this;
        for (char c : w.toCharArray()) {
            if (t.next[c - 'a'] == null) t.next[c - 'a'] = new Trie();
            t = t.next[c - 'a'];
        }
        t.end = true;
    }
    Trie walk(String s) {
        Trie t = this;
        for (char c : s.toCharArray()) { t = t.next[c - 'a']; if (t == null) return null; }
        return t;
    }
    boolean search(String w) { Trie t = walk(w); return t != null && t.end; }
    boolean startsWith(String p) { return walk(p) != null; }
}
```

START STATE · EVERY EXAMPLE BEGINS HERE

```
Trie t = new Trie();
t.insert("apple");
t.insert("app");
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`insert(String word)` RETURNS `void` O(length)	Adds the word, creating nodes along the way.	`t.insert("bat");` `boolean b = t.search("bat");  // → true`
`search(String word)` RETURNS `boolean` O(length)	True only for a whole word that was inserted.	`boolean a = t.search("app");  // → true` `boolean p = t.search("appl");  // → false`
`startsWith(String prefix)` RETURNS `boolean` O(length)	True if any inserted word starts with `prefix`.	`boolean p = t.startsWith("appl");  // → true` `boolean q = t.startsWith("b");  // → false`

BUILD IT YOURSELF

Fenwick tree (binary indexed tree)

3 entries

Prefix sums with point updates, both O(log n). Without updates, a plain prefix-sum array (`p[i + 1] = p[i] + a[i]`) answers range sums in O(1).

YOU WRITE THIS · THE EXAMPLES USE IT

```
static class Fenwick {
    long[] t;                                     // 1-based inside
    Fenwick(int n) { t = new long[n + 1]; }
    void add(int i, long delta) {                 // a[i] += delta, i is 0-based
        for (i++; i < t.length; i += i & -i) t[i] += delta;
    }
    long sum(int i) {                             // a[0] + ... + a[i - 1]
        long s = 0;
        for (; i > 0; i -= i & -i) s += t[i];
        return s;
    }
    long rangeSum(int l, int r) { return sum(r + 1) - sum(l); }   // a[l..r] inclusive
}
```

START STATE · EVERY EXAMPLE BEGINS HERE

```
Fenwick fw = new Fenwick(5);
int[] a = {5, 2, 7, 1, 3};
for (int i = 0; i < a.length; i++) fw.add(i, a[i]);
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`add(int i, long delta)` RETURNS `void` O(log n)	Adds `delta` to element i. To set a value, add the difference from the old one.	`fw.add(1, 10);` `long s = fw.rangeSum(1, 1);  // → 12`
`sum(int i)` RETURNS `long` O(log n)	Sum of the first i elements, a[0..i−1].	`long first3 = fw.sum(3);  // → 14`
`rangeSum(int l, int r)` RETURNS `long` O(log n)	Sum of a[l..r], inclusive.	`long mid = fw.rangeSum(1, 3);  // → 10`

Prefix sums when there are no updates

TEMPLATE

```
int[] a = {5, 2, 7, 1, 3};
long[] p = new long[a.length + 1];
for (int i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
long mid = p[3 + 1] - p[1];  // → 10
```

PART 8

Reading input and printing output

Scanner | BufferedReader and StringTokenizer | PrintWriter and fast output

Scanner

INPUT & OUTPUT

4 entries

The simplest way to read standard input on HackerRank-style platforms. Fine up to roughly 10^5 numbers; beyond that it is slow, so switch to `BufferedReader`.

IMPORT `java.util.Scanner`

START STATE · EVERY EXAMPLE BEGINS HERE

```
Scanner sc = new Scanner(System.in);   // input: "3 hello\n10 20 30\nlast line"
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE
`nextInt()` RETURNS `int` O(token)	Reads the next token as an `int`. Throws `InputMismatchException` if it is not a number, `NoSuchElementException` at end of input. Also `nextLong()` and `nextDouble()`.	`int n = sc.nextInt();  // → 3`
`next()` RETURNS `String` O(token)	Next whitespace-separated token.	`sc.nextInt();` `String w = sc.next();  // → "hello"`

`nextLine()` RETURNS `String` O(line)	The rest of the current line, without the newline. After `nextInt()` the rest of the line is often just `""`: call `nextLine()` once more to move to the next line.	`sc.nextInt();`
		`sc.next();`
		`String rest = sc.nextLine();  // → ""`
		`String line = sc.nextLine();  // → "10 20 30"`

`hasNext()` RETURNS `boolean` O(1)	True if another token exists; `hasNextInt()` and `hasNextLine()` check more precisely. Use them to read until end of input.	`int count = 0;`
		`while (sc.hasNextInt()) { sc.nextInt(); count++; }`
		`// count → 1`

BufferedReader and StringTokenizer

INPUT & OUTPUT

4 entries

Several times faster than `Scanner` for large input. `readLine()` returns whole lines; `StringTokenizer` splits a line into tokens.

IMPORT `java.io.*, java.util.StringTokenizer`

START STATE · EVERY EXAMPLE BEGINS HERE

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));   // input: "4 7\n1 2 3 4"
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`readLine()` RETURNS `String` O(line)	Next line without the newline, or `null` at end of input. Declares `IOException`, so the method must say `throws IOException`.	`String first = br.readLine();  // → "4 7"`
		`br.readLine();`
		`String none = br.readLine();  // → null`

`new StringTokenizer(String str)` RETURNS `StringTokenizer` O(1)	Splits on whitespace; `new StringTokenizer(str, ",")` for other delimiters.	`StringTokenizer st = new StringTokenizer(br.readLine());` `int n = Integer.parseInt(st.nextToken());  // → 4`
`nextToken()` RETURNS `String` O(token)	Next token; throws `NoSuchElementException` if none is left.	`StringTokenizer st = new StringTokenizer("a b");` `String x = st.nextToken();  // → "a"`

`hasMoreTokens()` RETURNS `boolean` O(1)	True if another token is left on this line.	`StringTokenizer st = new StringTokenizer("1 2 3");`
		`int sum = 0;`
		`while (st.hasMoreTokens()) sum +=`
		`Integer.parseInt(st.nextToken());`
		`// sum → 6`

Fast input template

TEMPLATE

```
static class FastReader {
    private final BufferedReader br;
    private StringTokenizer st;
    FastReader(InputStream is) { br = new BufferedReader(new InputStreamReader(is)); }
    String next() throws IOException {
        while (st == null || !st.hasMoreTokens()) {       // while, not if: skips blank lines
            String line = br.readLine();
            if (line == null) return null;                // end of input
            st = new StringTokenizer(line);
        }
        return st.nextToken();
    }
    int nextInt() throws IOException { return Integer.parseInt(next()); }
    long nextLong() throws IOException { return Long.parseLong(next()); }
}

FastReader in = new FastReader(new ByteArrayInputStream("3\n5 1 4".getBytes()));   // in a real solution: new FastReader(System.in)
int n = in.nextInt();
long total = 0;
for (int i = 0; i < n; i++) total += in.nextInt();
// total → 10
```

INPUT & OUTPUT

PrintWriter and fast output

♥ 4 entries

`System.out.println` flushes often and is slow for many lines. Collect output in one `PrintWriter` (or a `StringBuilder`) and flush once at the end.

IMPORT `java.io.*`

START STATE · EVERY EXAMPLE BEGINS HERE

```
PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
```

METHOD · RETURNS · COST	WHAT IT DOES	EXAMPLE

`println(x)`
TAKES `any type`
RETURNS `void` `O(length)`
	Prints `x` and a newline; `print(x)` without the newline. Accepts any type.	`out.println(42);`
		`out.print("a");`
		`out.print(' ');`
		`out.println(3.5);`

`printf(String format, Object... args)` RETURNS `PrintWriter` `O(length)`	Formatted output, same codes as `String.format`.	`out.printf("%d %.3f%n", 7, Math.PI);  // prints "7 3.142"`
`flush()` RETURNS `void` `O(buffer)`	Writes the buffered text out. Forget it (or `close()`) and nothing is printed at all.	`out.flush();`

`StringBuilder + one print`
TAKES —
RETURNSnot a call `O(total)`
	Build all lines first and print once: as fast as `PrintWriter`.	`StringBuilder sb = new StringBuilder();`
		`for (int i = 1; i <= 3; i++)`
		`sb.append(i).append('\n');`
		`out.print(sb);`

NOTES

Traps, one line each

check before you submit

The mistakes that compile fine and then fail hidden test cases. Each links to the section that explains it.

- `int` sums and products overflow silently: use `long`, and cast *before* multiplying: `(long) a * b`. **Arithmetic**
- `%` keeps the sign of the left operand: use `Math.floorMod` for negative numbers. **Math**
- Comparators written as `a - b` can overflow: use `Integer.compare(a, b)`. **Comparator**
- `==` on `Integer`, `Long` or `String` compares references: use `equals`, or unbox first. **ArrayList**
- `list.remove(3)` removes index 3; `list.remove(Integer.valueOf(3))` removes the value 3. **ArrayList**
- `map.get(k)`, `floorKey`, `ceiling`, `poll` and `peek` return `null` when there is nothing; unboxing that into `int` throws. **HashMap**
- `PriorityQueue.remove(x)` and `contains(x)` are O(n), and printing a heap does not show sorted order. **PriorityQueue**
- A `TreeSet` or `TreeMap` with a comparator treats "compares as 0" as a duplicate and drops it. **TreeSet**
- `int[]` keys in a `HashMap` / `HashSet` compare by reference: use a `List`, a `String` or an encoded `long`. **HashMap**
- Changing a collection inside a for-each loop over it throws `ConcurrentModificationException`: use `removeIf` or an `Iterator`. **Iterator**
- `s += x` in a loop is O(n²): use `StringBuilder`. `split` takes a regex: escape `.` and `|`. **String**
- `result.add(path)` in backtracking stores a reference: add `new ArrayList<>(path)`. **ArrayList**
- `ArrayDeque` rejects `null`: check children before `offer` in tree BFS. **Stack**
- Recursion deeper than about 10^4 frames can overflow the stack: switch to an explicit stack. **Graphs**
- `1 << k` with $k \geq 31$ is wrong: use `1L << k`. **Bits**
- `Scanner.nextInt()` followed by `nextLine()` returns the empty rest of the line; and output in a `PrintWriter` is lost without `flush()`. **Scanner**
- `List.of`, `Map.of`, `String.repeat`, `isBlank`, `var` and `toList()` need Java 9–16; if the platform runs Java 8 they do not compile. **Version tags**

Made with care for Purvam · 391 methods & idioms · 576 example results checked on OpenJDK 25.0.2 · good luck on 6 December